

Table of Contents

.....	2
Introduction to Disk and Forensic Imaging	3
1. Partition Schemes	3
2. FAT32 File System	3
.....	3
3. NT File System.....	3
4. Disk Forensic Images	3
5. Memory Images	3
TASK 1 - Process MetaData and Memory Acquisition 1.1 Indroduction to Task.....	4
1.2 Methods ands Tools.....	4
.....	4
1.3 Relevant Data Identified.....	4
Part 1 - Process Metadata	4
Part 2 – Memory Acquisition	6
Part 3 – Hash Verification	7
TASK2 - FAT32 Volume Creation and Forensic Imaging	8
2.1 Introduction to Task	8
2.2 Method & Tools	8
2.3 Relevant Data Identified	8
Part 2A - VHD Creation with FAT32 and MBR.....	8
Part 2B - File Properties.....	11
Part 2C - Disk Geometry and Sectors Per Cluster.....	12
Part 2D - Volume Serial Number.....	13
Part 2E - Forensic Image Acquisition.....	14
Task 3.....	16
3.1 Introduction to Task.....	16
3.2 Method & Tools.....	16
Part 3A - Directory Entry.....	16
Part 3B - File Attributes.....	18
Part 3C - Starting Cluster Number.....	19
Part 3D - File Date and Times.....	19
Part 3E - FAT Entries and Cluster Chain.....	19
.....	20
Part 3F - File Deletion.....	20
Task 4 – NTFS Volume Creation and Forensic Imaging.....	23
4.1 Introduction to Task.....	23
4.2 Methods and tools.....	23
4.3 Relevant Data Identified.....	24
Part 4A - NTFS Volume Creation.....	24
.....	25
Part 4B - File Properties.....	25
Part 4C – Selective Forensic Image Acquisition.....	27
Part 4D - Post-Acquisition Actions.....	29
Part 4E - Logical Copy vs Selective Copy.....	29
TASK 5 5.1 Introduction to Task.....	29
.....	30
5.3 Relevant Data Identified.....	30
Part 5A - MFT Record IDs and Parenting.....	30
Part 5B - MAC Time Analysis and Anomalies.....	31
Part 5C – How NTFS Differentiates Files and Directories.....	32

Part 5D - NTFS Volume Serial Number and Physical/Logical Anomaly.....	32
.....	34
Part 5E - Running and Terminated Processes.....	34
Task-6.....	36
6.1 Introduction to Task	36
6.2 Method and Tools	36
6.3 Relevant Data Identified	37
Part 6A – Network Configuration	37
Part 6B – Network Connections Including TCP	38
Part 6C – Firewall Rules and Exceptions	39
Part 6D – RDP Remote Access Information and Status	40
Part 6E – Network Shares	41
References.....	42

Introduction to Disk and Forensic Imaging

This report documents the forensic analysis conducted on the CFL Cyber virtual machine, carried out in accordance with the ACPO Good Practice Guide for Digital Evidence (ACPO, 2012), the Forensic Science Regulator's Codes of Practice (FSR, 2023), ISO/IEC 27037:2012, and ISO/IEC 17025:2017 (ISO, 2012; ISO, 2017).

1. Partition Schemes

A partition scheme defines how a storage medium is logically divided. The Master Boot Record (MBR), used in Task 2, stores the partition table in sector 0 and supports up to four primary partitions and a maximum disk size of 2 TB. The GUID Partition Table (GPT), used in Task 4, addresses these limitations by storing primary and backup partition tables at both ends of the disk and supporting up to 128 partitions (Carrier, 2005).

2. FAT32 File System

FAT32 organises a volume into a reserved region containing the boot sector and BIOS Parameter Block, the File Allocation Table itself, the root directory, and the data region. Directory entries are 32 bytes and include the 8.3 filename, attributes, timestamps, starting cluster and file size, with Long File Name entries (attribute 0x0F) supporting filenames beyond 8.3. On deletion, only the first byte of the directory entry is overwritten with 0xE5 and the FAT chain is cleared, leaving file data recoverable until overwritten (Nelson, Phillips and Steuart, 2018).

3. NT File System

NTFS represents every file and directory in the Master File Table (MFT) as a 1,024-byte record composed of attributes including \$STANDARD_INFORMATION, \$FILE_NAME and \$DATA. Directories instead use \$INDEX_ROOT and \$INDEX_ALLOCATION, which is how NTFS differentiates the two. The \$FILE_NAME attribute stores a reference to the parent directory's MFT record, enabling full-path reconstruction from any record (Carrier, 2005).

4. Disk Forensic Images

A forensic image is a bit-for-bit copy that preserves allocated data, unallocated space, slack and file system metadata, ensuring the original is not altered (ACPO Principle 1). Images may be physical, logical or selective; selective copies capture only specified items and therefore exclude unallocated space (Casey, 2011). Integrity is verified through cryptographic hashing, with the E01 container embedding hashes and case metadata as required by ISO/IEC 27037 (ISO, 2012).

5. Memory Images

A memory image captures the volatile contents of RAM, which is lost once power is removed. Memory may contain running and terminated process structures, network sockets, and credentials

that never reach disk, and is typically acquired alongside the pagefile to recover swapped content (Ligh et al., 2014). Acquisition follows the order of volatility (Brezinski and Killalea, 2002), with hashing performed immediately to preserve integrity.

TASK 1 - Process MetaData and Memory Acquistion

1.1 Indroduction to Task

Task 1 requires the examiner to open a command prompt (cmd.exe), minimise the window, and use a separate console tool to retrieve process metadata including file name, Process ID (PID), execution date and time, and the full executable path. The task further required acquisition of a physical memory dump and the virtual memory (pagefile) using a forensic tool, followed by cryptographic hashing to establish integrity. This task adheres to ACPO Principle 3, requiring an audit trail of processes applied to digital evidence, and ISO/IEC 27037:2012 guidance on the acquisition of volatile data (ACPO, 2012; ISO, 2012).

1.2 Methods ands Tools

Tool	Purpose
Cmd.exe	Process under investigation
Windows Powershell	Retrieval of process metadata
FTK Imager (v4.x)	Physical memory acquisition and hash verification

The following tools were used to complete task 1 ;
Step 1: cmd.exe was launched and minimised. PowerShell was then used to query process metadata using the Get-Process and Get-WmiObject cmdlets.

Step 2: FTK Imager was launched with Administrator privileges. The 'Capture Memory' function was used to acquire a physical memory dump (memdump.mem) with the pagefile included.

Step 3: FTK Imager's 'Verify Drive/Image' function was used to compute and verify hash values. PowerShell's Get-FileHash cmdlet was additionally used to independently verify the MD5 and SHA-1 hash values.

1.3 Relevant Data Identified

Part 1 - Process Metadata

The following process metadata was retrieved for the cmd.exe process using PowerShell (Get-WmiObject Win32_Process):

Field	Value
File Name	cmd.exe
Process ID (PID)	7420
Executable Path	C:\Windows\System32\cmd.exe
Start Time (Get-Process)	4/1/2026 11:31:14 AM
Creation Date (WMI)	20260401113114.136157+060

```
Administrator: Windows PowerShell

PS C:\WINDOWS\system32> Get-Process -Name cmd | Select-Object ID, Name, Path, StartTime

Id Name Path                               StartTime
-- --
7420 cmd C:\WINDOWS\system32\cmd.exe 4/1/2026 11:31:14 AM

PS C:\WINDOWS\system32>
```

Figure 1: PowerShell Get-Process output confirming executable path

```
PS C:\WINDOWS\system32> Get-WmiObject win32_Process -Filter "name='cmd.exe'" | Select-Object Name, ProcessID, ExecutablePath, CreationDate

Name      ProcessID ExecutablePath          CreationDate
-----
cmd.exe    7420      C:\WINDOWS\system32\cmd.exe 20260401113114.136157+060

PS C:\WINDOWS\system32>
```

Figure 2: PowerShell Get-WmiObject output showing cmd.exe process metadata

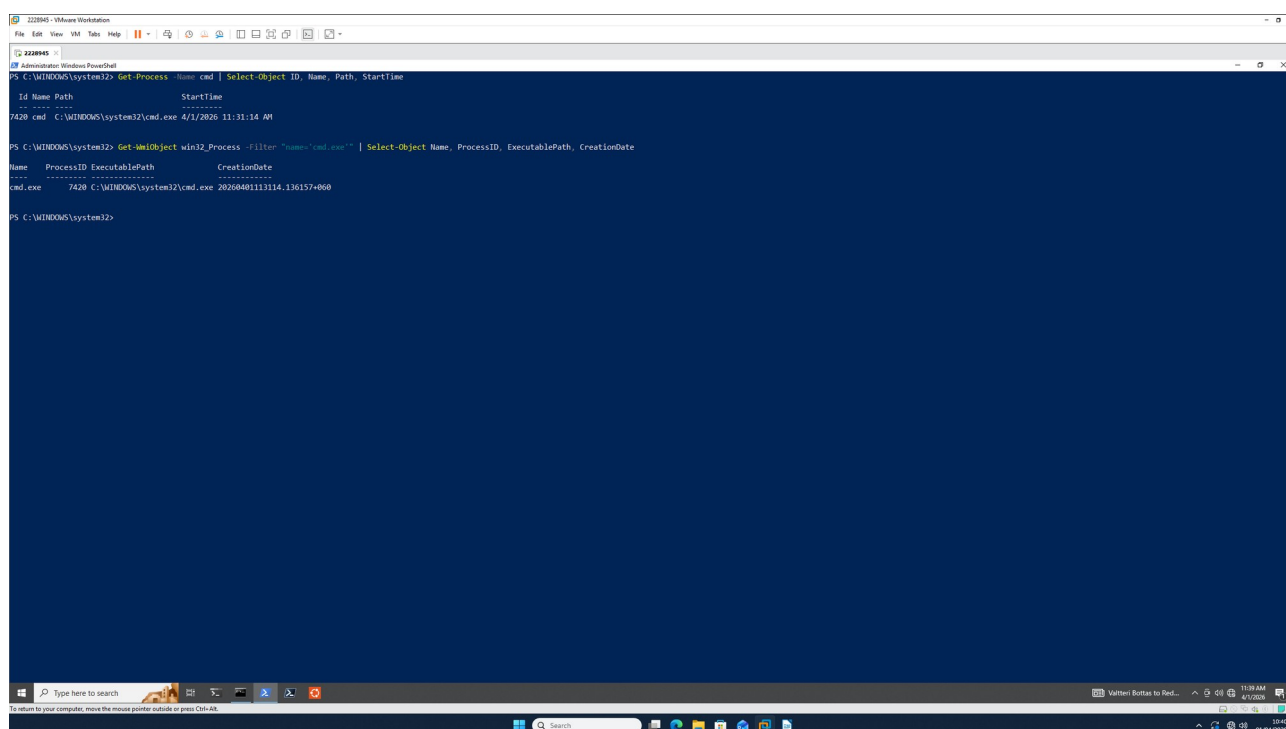


Figure 3: Full Screenshot with timestamp

The WMI CreationDate value 20260401113114.136157+060 decodes to 1 April 2026 at 11:31:14.136157 in the UTC+60 minute timezone, confirming the Get-Process StartTime value and demonstrating consistency between the two independent retrieval methods.

Part 2 – Memory Acquisition

FTK Imager was used to capture physical memory to a file named memdump.mem. The pagefile (virtual memory) was captured simultaneously using the 'Include pagefile' option during acquisition.

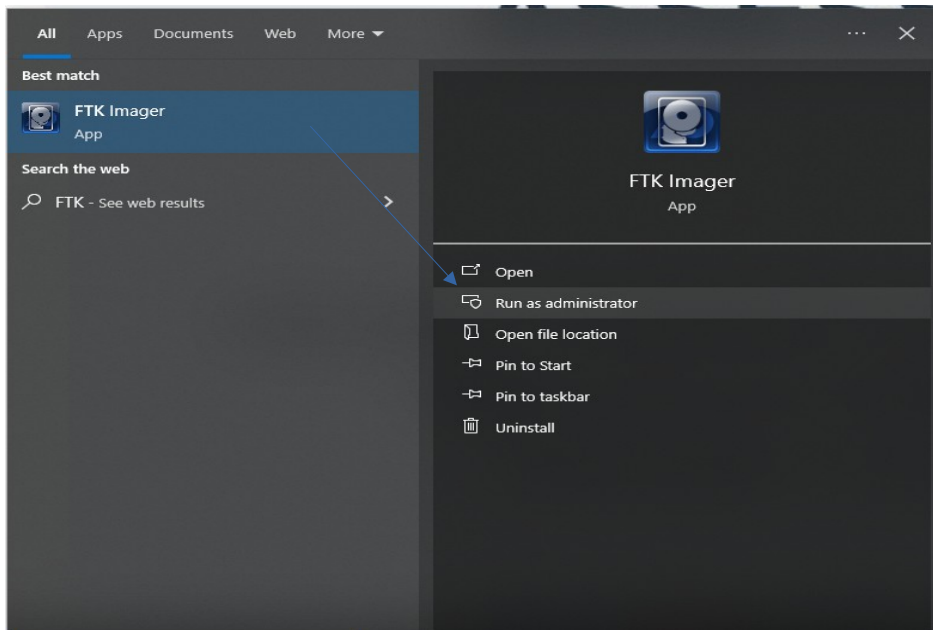


Figure 2: Running FTK Imager as Administrator

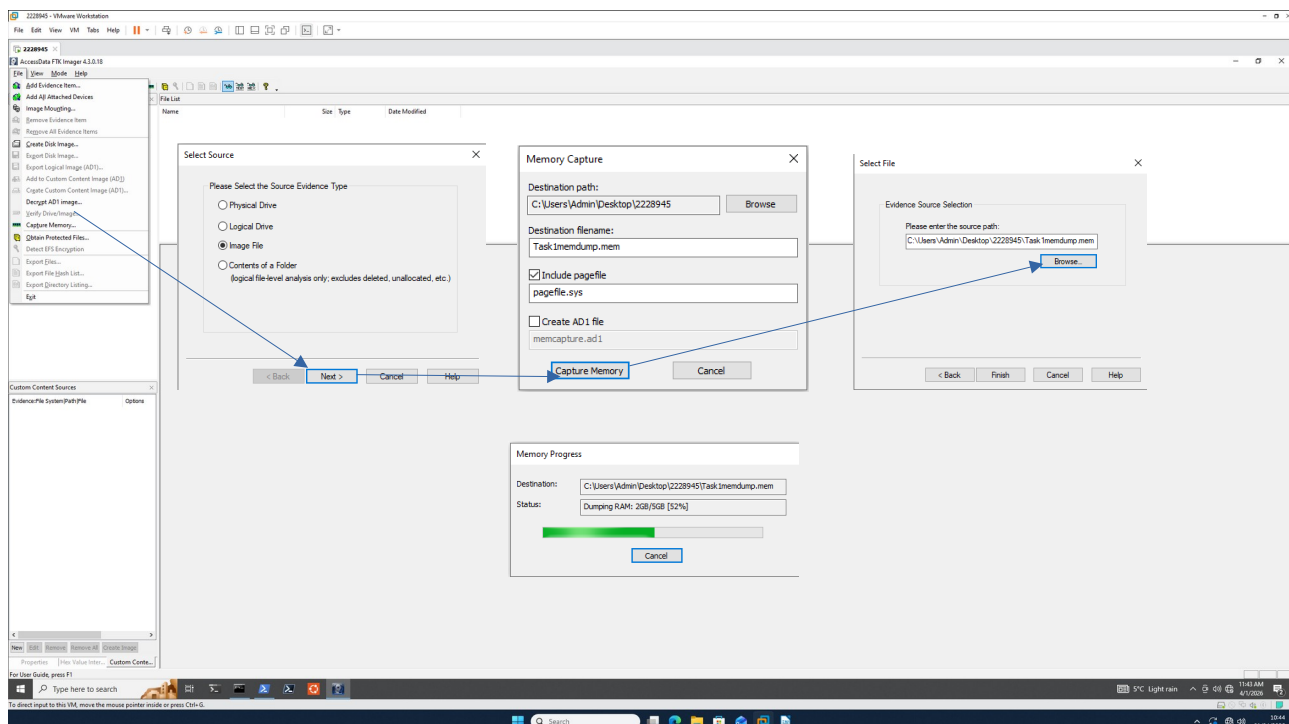


Figure 3: FTK Imager Memory Capture dialog and memory acquisition in progress

Part 3 – Hash Verification

Following acquisition, FTK Imager's Drive/Image Verify function was used to compute hash values and verify their integrity. The following hash values were recorded:

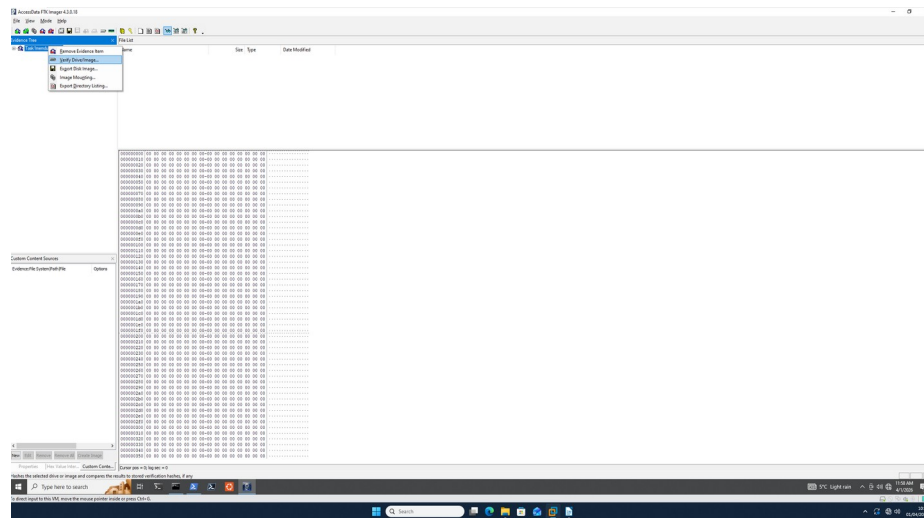


Figure 4: Using FTK Imager to verify hash

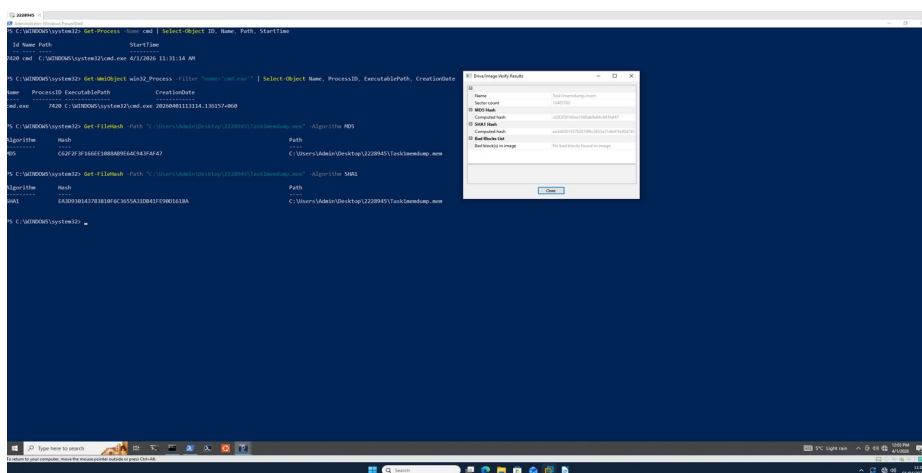


Figure 5: PowerShell Get-FileHash output for memory dump and FTK Imager Drive/Image Verify Results showing MD5 and SHA1 hashes

To provide independent verification, PowerShell's Get-FileHash cmdlet was used to recompute the hash values of the memory dump

Hash Algorithm	Value
MD5	C62F2F3F166EE1088AB9E64C943FAF47
SHA-1	EA3D930143783810F6C3655A31DB41FE90D1618A

The hash values produced by FTK Imager and PowerShell are identical for both MD5 and SHA-1, confirming through two independent tools that the memory image was not altered between acquisition and verification. FTK Imager additionally reported no bad blocks within the image. In a full investigation the pagefile (pagefile.sys) would be independently hashed using the same process; the pagefile was captured in this task but was not separately hashed, and this would be actioned as a

post-acquisition step prior to analysis to provide complete evidential coverage of both volatile artefacts.

Hashing is a core principle of digital forensic evidence handling. A cryptographic hash function produces a fixed-length digest that uniquely represents the contents of a file at a given moment; altering any single bit changes the digest substantially, allowing examiners to prove that evidence has not been modified between acquisition and analysis (Casey, 2011). SHA-256 is considered computationally unique as the probability of two different inputs producing the same digest is negligible. MD5 and SHA-1 have known theoretical collision vulnerabilities but remain widely accepted in established forensic tools and courts when used for integrity verification of a known input, as collision resistance differs from integrity verification (Nelson, Phillips and Steuart, 2018). The use of two algorithms and two tools, as performed here, strengthens the evidential position by reducing reliance on any single algorithm or implementation.

TASK2 - FAT32 Volume Creation and Forensic Imaging

2.1 Introduction to Task

Task 2 required the examiner to create a 5GB Virtual Hard Disk (VHD) formatted with a FAT32 file system using an MBR partition scheme. A file exceeding 200KB was to be saved to the volume and its properties recorded. The task also required identification of the Volume Serial Number in PowerShell, its physical location within the file system, disk geometry data including sectors per cluster, and the creation of a forensic copy of the volume with hash verification.

2.2 Method & Tools

Tool	Purpose
DiskPart (built-in)	VHD creation, MBR partitioning, FAT32 formatting
Windows PowerShell	File properties, volume info, partition info, serial number
fsutil.exe (built-in)	Sector info, volume info, file system statistics
FTK Imager	Forensic image acquisition, hash verification, boot sector analysis

2.3 Relevant Data Identified

Part 2A - VHD Creation with FAT32 and MBR

A 5GB fixed-size Virtual Hard Disk (VHD) was created using Disk Management and initialised using a Master Boot Record (MBR) partition scheme. The volume was formatted with the FAT32 file system and assigned the volume label '2228945'. The MBR partition scheme was selected as it is compatible with FAT32 and is the conventional scheme used for volumes under 2TB. The MBR is located at sector 0 of the disk and contains the partition table, which defines the size and location of partitions on the volume.

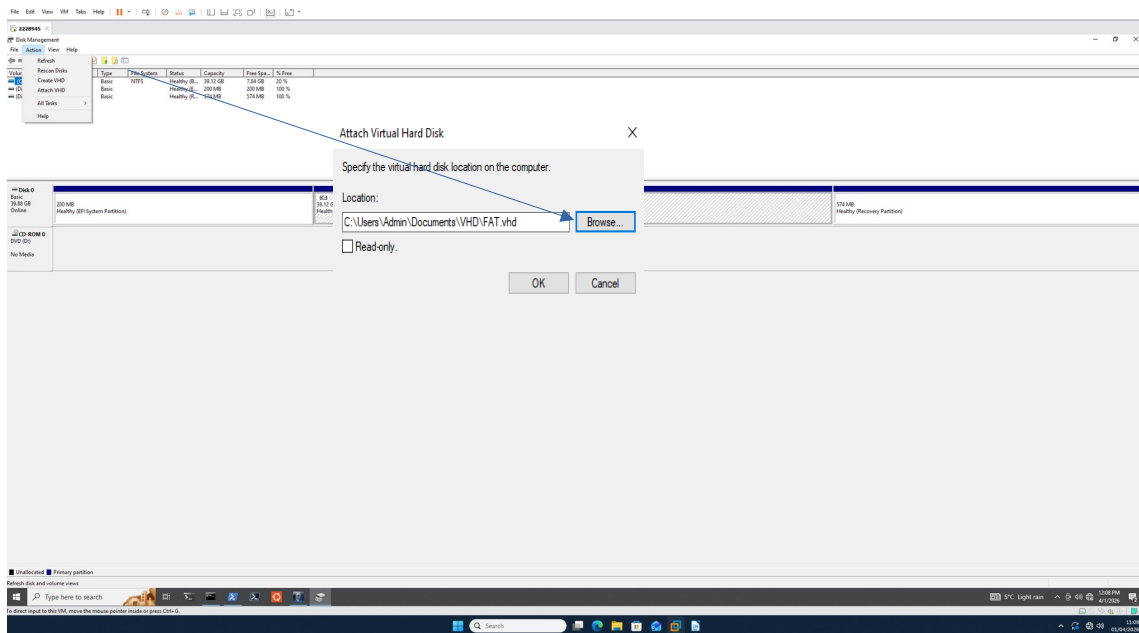


Figure 6: Disk Management showing VHD creation dialog

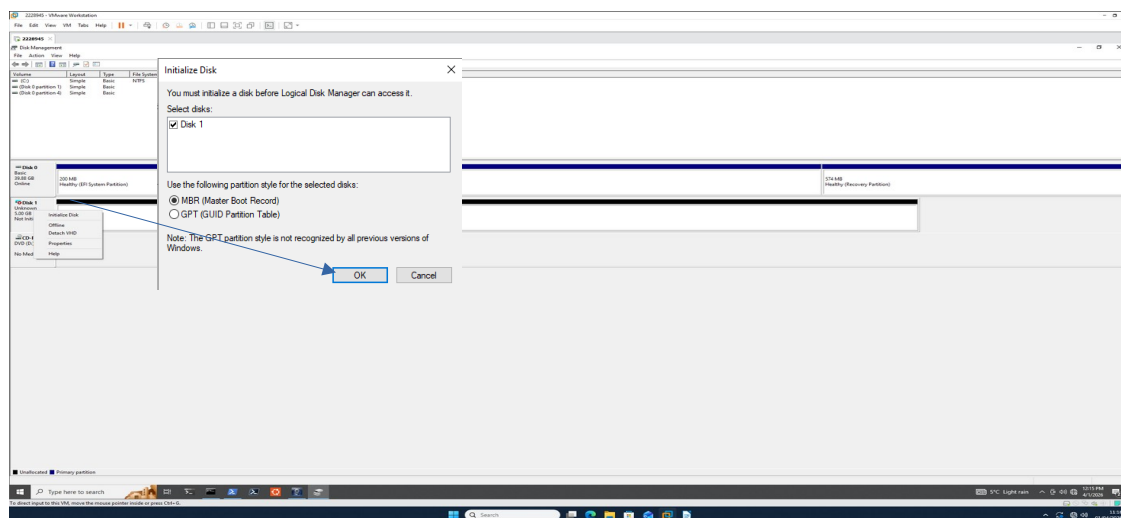


Figure 7: MBR initialisation dialog

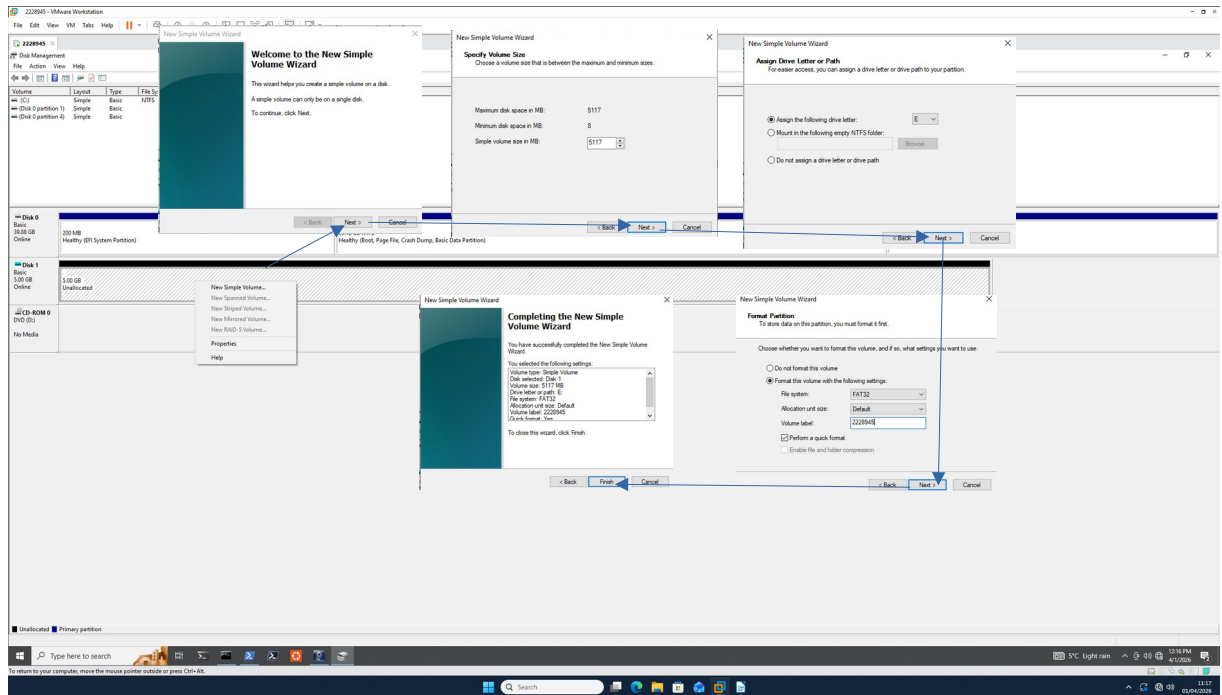


Figure 9: FAT32 format dialog showing volume label 2228945

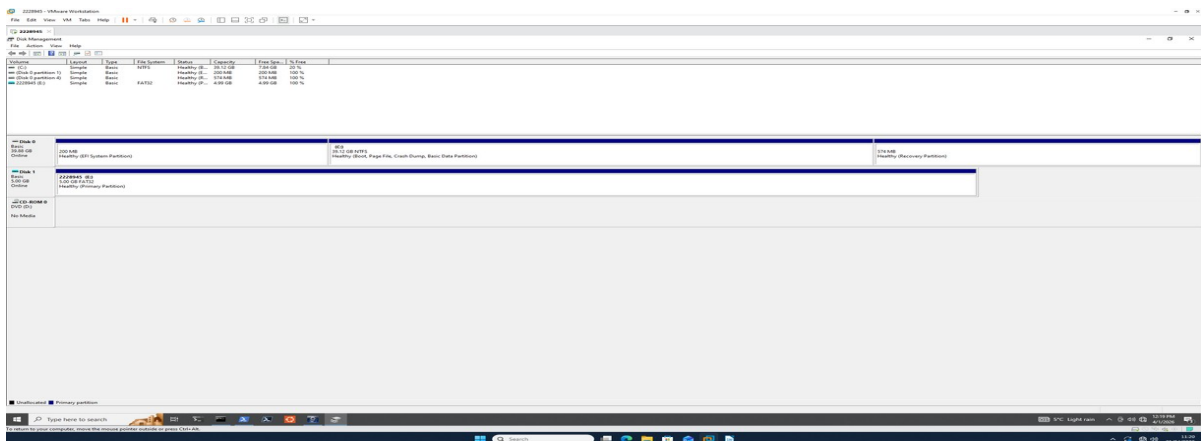


Figure 8: Disk Management showing completed FAT32 volume

Part 2B - File Properties

A screenshot file exceeding 200KB was copied to the FAT32 volume using the PowerShell Copy-Item cmdlet. The use of a built-in console tool ensures the copy process is documented and reproducible. The following file properties were recorded:

Property	Value
Name	Screenshot (1).png
Full Path	E:\Screenshot (1).png
Logical Size	218,973 bytes
Creation Time	4/1/2026
Last Modified	4/1/2026
Last Accessed	4/1/2026
Attributes	Archive

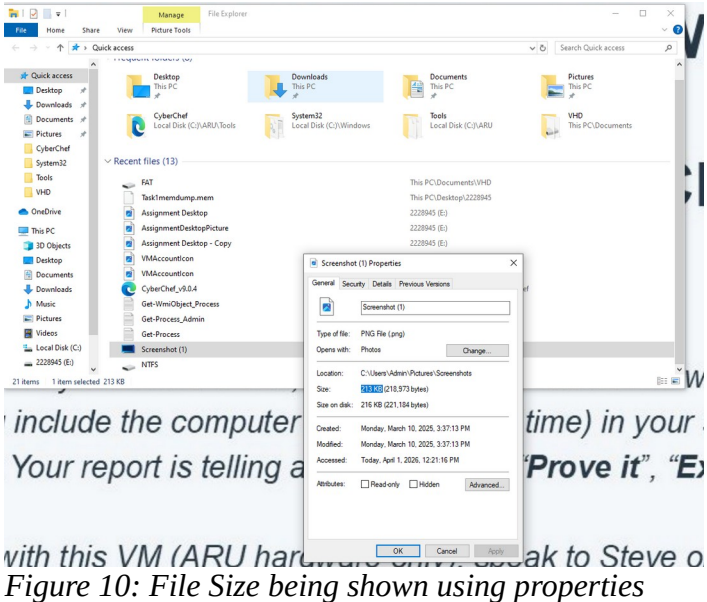


Figure 10: File Size being shown using properties

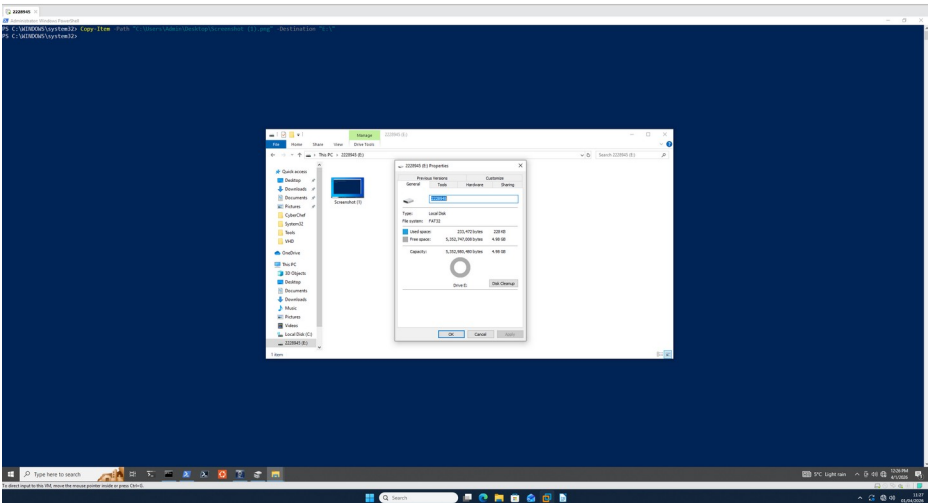


Figure 11: PowerShell Copy-Item command copying file to FAT32 volumeFile and Explorer showing file on FAT32 volume with Properties window

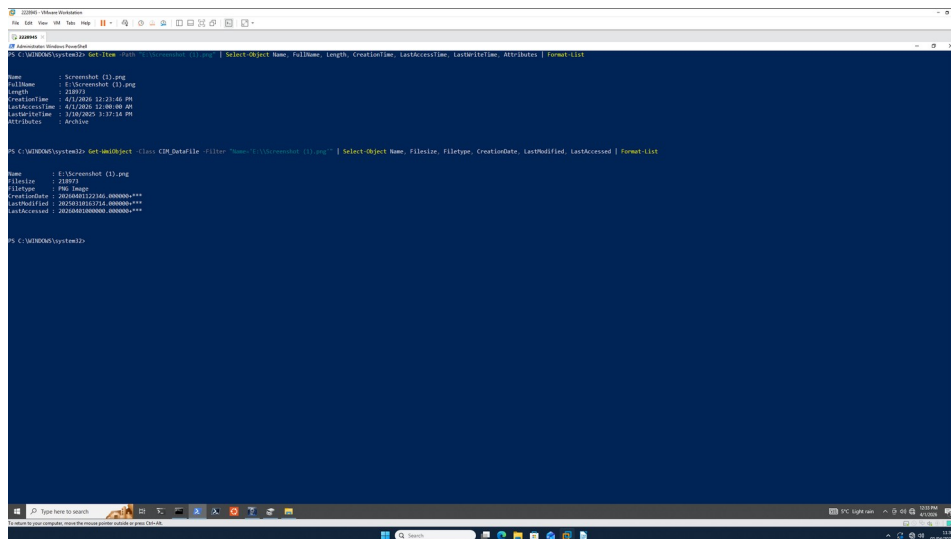


Figure 12: PowerShell Get-Item output showing file properties and Get-WmiObject CIM_DataFile output showing logical size

Part 2C - Disk Geometry and Sectors Per Cluster

The disk geometry of Partition 1 was identified using the Get-WmiObject Win32_DiskDrive cmdlet

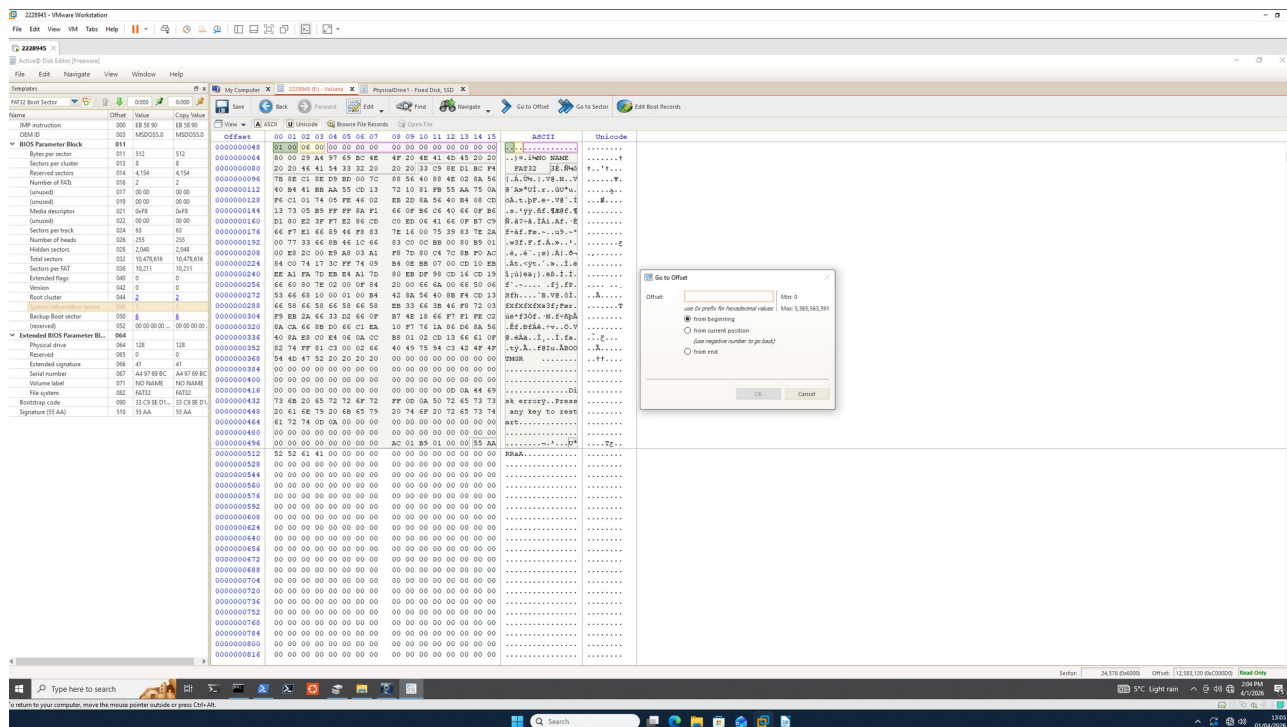


Figure 13: Active Disk Editor showing FAT32 Boot Sector BIOS Parameter Block displaying disk geometry including bytes per sector (512), sectors per cluster (8) and total sectors (10,479,616) in PowerShell. Disk geometry describes the physical structure of a storage device in terms of cylinders, heads, tracks and sectors. This information is stored within the Master Boot Record (MBR) at sector 0 of the disk.

The sectors per cluster value was identified using the `fsutil fsinfo volumeinfo` command. The following values were recorded:

Property	Value
Bytes Per Sector	512
Sectors Per Cluster	8
Cluster Size	4,096 bytes
Total Sectors	10,479,616
Reserved sectors	4,154
Number of FATs	2
Sectors per FAT	10,211
Root cluster	2
Backup Boot sector	6

The cluster is the smallest unit of allocation on a FAT32 volume. Each cluster consists of 8 sectors $\times$ 512 bytes = 4,096 bytes. Files are allocated in whole clusters, meaning a file smaller than 4,096 bytes will still occupy one full cluster, resulting in slack space.

```

C:\WINDOWS\system32> Get-WmiObject -Class WinDisk -Namespace \\.\root\cimv2 -Filter "VolumeName='Windows' OR VolumeName='OS'" | Format-List

Model : Microsoft Virtual Disk
TotalSectors : 452
TotalHeads : 255
TotalTracks : 3685440
TotalCylinders : 166208
CylinderPerTrack : 255
SectorPerTrack : 1
SectorPerCylinder : 1

Model : VMware Virtual HDD Disk
TotalSectors : 5221
TotalHeads : 255
TotalTracks : 8197345
TotalCylinders : 133155
CylinderPerTrack : 255
SectorPerTrack : 1
SectorPerCylinder : 1

C:\WINDOWS\system32> fsutil info volume{F:}

Volume Name : Z:\0005
Volume Serial Number : 0dc6997d
Maximum Component Length : 255
File System Name : FAT32
Number of Sectors : 1
Thinly-Provisioned :
Supports Case of Filenames :
Supports Unicode in Filenames :
Supports Hard-Link-File Information :
Supports Encrypted File System :

```

Figure 14: PowerShell Get-WmiObject Win32_DiskDrive output showing disk geometry and fsutil fsinfo volumeinfo output showing sectors per cluster

Part 2D - Volume Serial Number

The Volume Serial Number (VSN) was retrieved using the Get-WmiObject Win32_LogicalDisk cmdlet in PowerShell. The VSN identified was **BC6997A4**.

The VSN was then located within the FAT32 Boot Sector using Active Disk Editor. The VSN is stored at offset **0x43** (67 bytes from the start of the boot sector) as a 4-byte value in **little endian** format. This means the bytes are stored in reverse order on disk:

Source	Value
PowerShell (logical)	BC6997A4
Active Disk Editor (physical)	A4 97 69 BC

The screenshot shows a Windows 10 desktop environment. A PowerShell window is open, displaying the following command and output:

```
PS C:\WINDOWS\system32> Get-Object $WMKODSystem32 | Select-Object VolumeSerialNumber | Format-List
VolumeSerialNumber : EC595764
```

The taskbar at the bottom of the screen includes the Start button, a search bar, and several pinned applications: File Explorer, Edge, Task Manager, and the Windows Security app. The system tray on the right shows the date and time as 12:42 PM on 4/10/2020.

Figure 15: PowerShell Get-WmiObject output showing Volume Serial Number BC6997A4

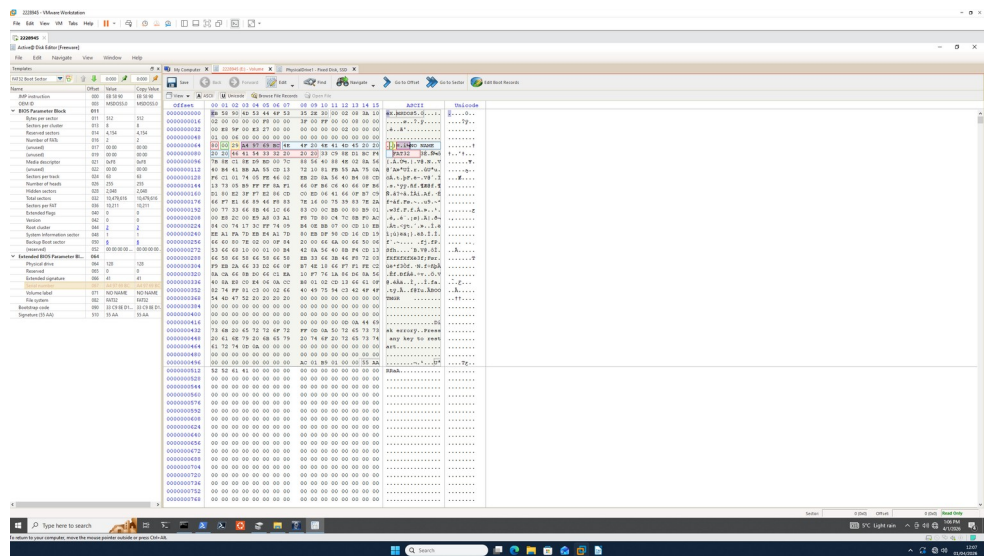


Figure 16: Active Disk Editor showing FAT32 boot sector with VSN at offset 0x43

Part 2E - Forensic Image Acquisition

A forensic image of the FAT32 volume was acquired using FTK Imager (v4.3.0.18) in E01 format. The E01 format was selected as it is an industry standard forensic container that supports built-in hash verification and case metadata. The following case details were recorded during acquisition:

Field	Value
Case Number	2228945
Evidence Number	1

Description	TASK2
Examiner	Steve
Source Data Size	5,117 MB
Acquisition Started	Wed Apr 1 13:26:33 2026
Acquisition Finished	Wed Apr 1 13:27:22 2026

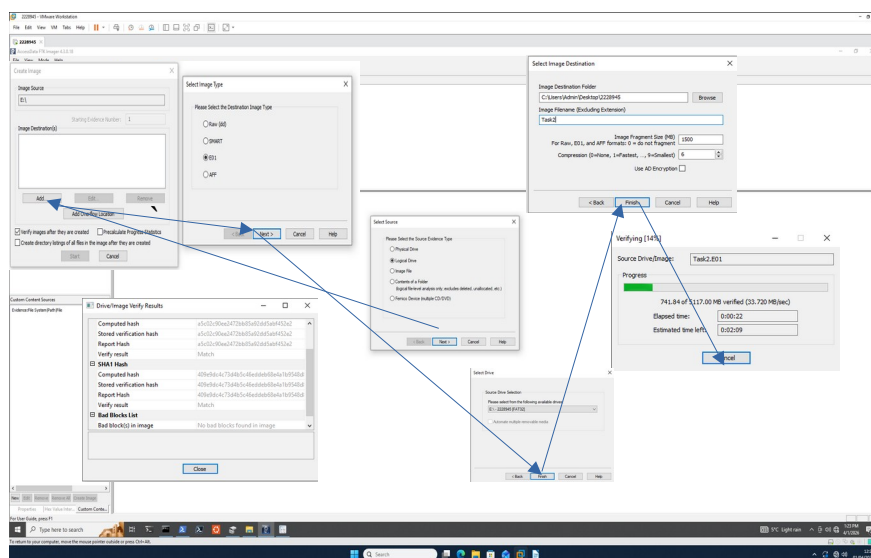


Figure 17: FTK Imager Create Disk Image settings showing E01 format selected and Verify Results showing MD5 and SHA1 hashes verified

Following acquisition FTK Imager automatically verified the image by recomputing the hash values and comparing them against those recorded during acquisition:

Hash Algorithm	Value	Result
MD5	a5c02c90ee2472bb85a92dd5abf452e2	Verified
SHA1	409e9dc4c73d4b5c46eddeb68e4a1b9548d87690	Verified

The computed hashes matched the stored verification hashes, confirming the forensic image is a bit-for-bit identical copy of the original FAT32 volume. No bad blocks were identified in the image.

This process adheres to **ACPO Principle 2**, which states that any person accessing digital evidence must be competent to do so and must be able to explain the relevance and implications of their actions. By using FTK Imager with hash verification, the examiner can demonstrate that the original evidence has not been altered during the acquisition process (ACPO, 2012).

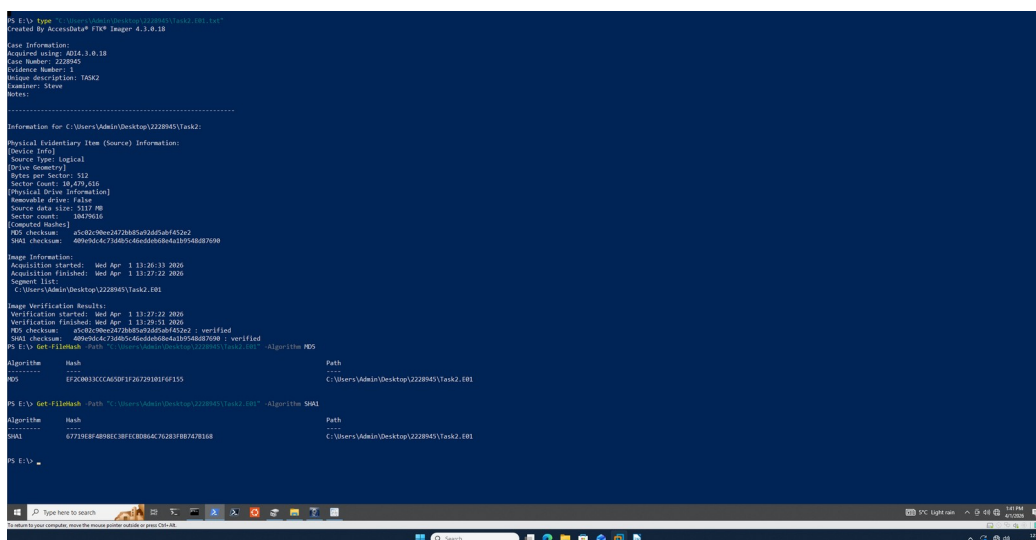


Figure 18: PowerShell Get-FileHash output showing hashes of Task2.E01 and FTK Imager case report (Task2.E01.txt) showing acquisition details

Task 3

3.1 Introduction to Task

Task 3 required the examiner to locate the file saved to the FAT32 volume in Task 2 and examine its directory entry. This included identifying all relevant metadata fields, the file attributes, the starting cluster number, the file date and times, and the relevant FAT entries showing which clusters the file content is stored across. Additionally, the task required a demonstration of what occurs to the directory entry when the file is deleted from the volume.

3.2 Method & Tools

Tool	Purpose
Active Disk Editor	Examination of raw directory entry, FAT entries and hex data
Windows Explorer	File deletion demonstration
Windows PowerShell	Supporting file property verification

Part 3A - Directory Entry

The directory entry for the file Screenshot (1).png was located within the Root Directory of the FAT32 volume using Active Disk Editor. FAT32 directory entries are fixed at 32 bytes in length and store all metadata associated with a file.

As the filename Screenshot (1).png exceeds the 8.3 naming convention supported by FAT32, the operating system automatically generated a Short File Name (SFN) of **SCREEN~1.PNG**. Additionally, Long File Name (LFN) entries were created preceding the SFN entry to store the full filename in Unicode format. The LFN entries are identified by an attribute byte value of **0x0F**.

The following metadata fields were identified from the directory entry:

Field	Offset	Value	Explanation
File Name	0x00	SCREEN~1	8.3 short filename
Extension	0x08	PNG	File extension
Attributes	0x0B	0x20	Archive flag set
Creation Time	0x0E	62 81	Time value
Creation Date	0x10	5C 81	Date value
Last Accessed Date	0x12	5C 00	Date only (no time in FAT32)
High Cluster	0x14	00 00	Upper 16 bits of cluster
Modified Time	0x16	84 6A	Time value
Modified Date	0x18	5A 05	Date value
Low Cluster	0x1A	05 00	Starting cluster = 5
File Size	0x1C	218,973 bytes	Logical file size

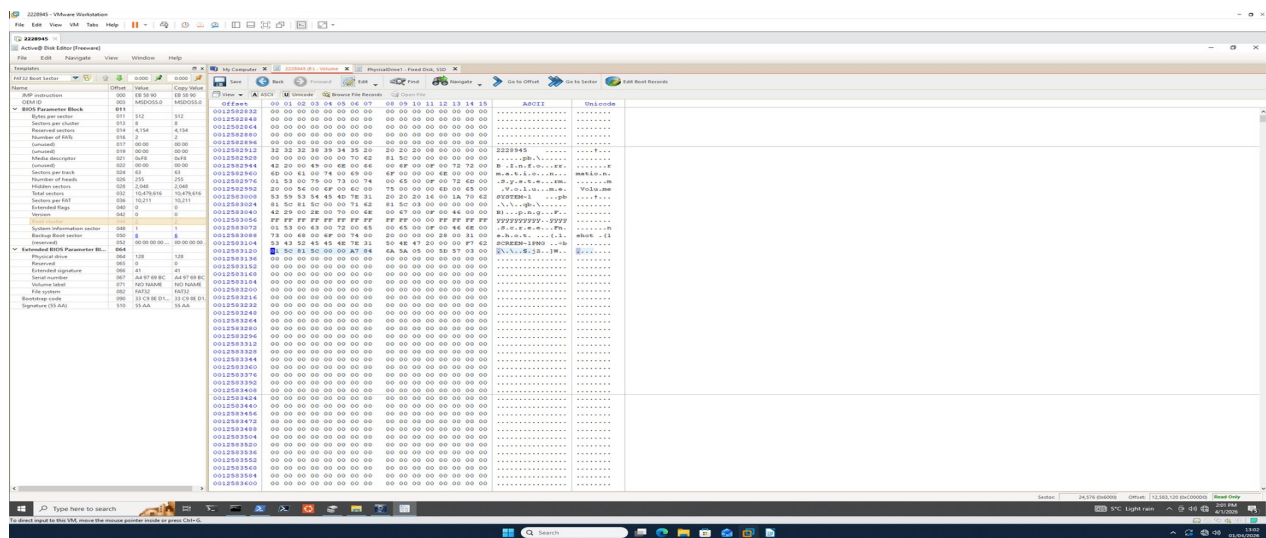


Figure 19: Active Disk Editor showing Root Directory of FAT32 volume with LFN (Long File Name) entries visible above the SCREEN~1.PNG Short File Name directory entry

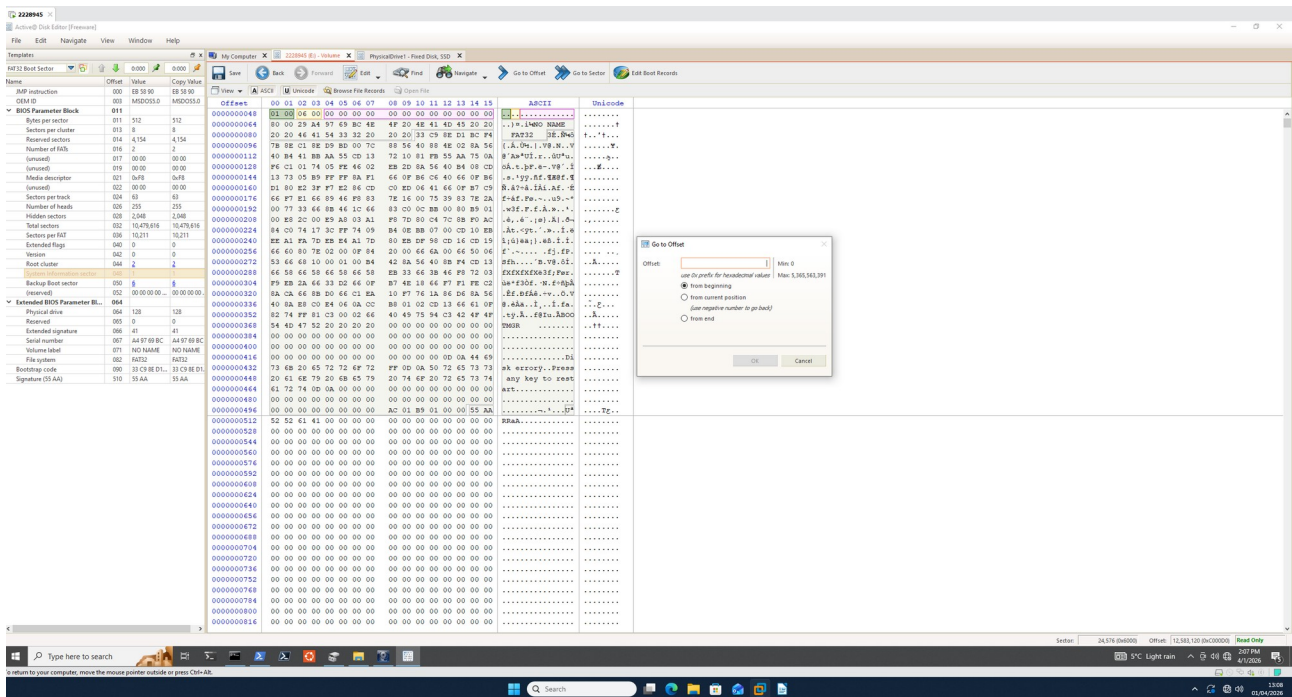


Figure 20: Active Disk Editor showing the full 32-byte directory entry for SCREEN~1.PNG including filename, extension, attributes, creation date, last accessed date, modified date, starting cluster and file size fields

Part 3B - File Attributes

The Attributes byte at offset 0x0B contains the value 0x20, which indicates the Archive attribute is set. This is the standard attribute applied to regular user files and indicates the file has been created or modified since the last backup. The archive attribute is commonly used by backup software to identify files requiring backup.

Attribute Value	Meaning
0x01	Read Only
0x02	Hidden
0x04	System
0x08	Volume Label
0x0F	LFN Entry
0x10	Directory
0x20	Archive

Part 3C - Starting Cluster Number

The starting cluster number is calculated by combining the High Cluster (offset 0x14) and Low Cluster (offset 0x1A) values:

- High Cluster = 0x0000
- Low Cluster = 0x0005
- Starting Cluster = 5

This means the file data begins at cluster 5 on the FAT32 volume. The root directory of a FAT32 volume always begins at cluster 2, with user data starting from cluster 3 onwards.

Part 3D - File Date and Times

Three timestamps were identified in the directory entry:

Timestamp	Value	Explanation
Created	4/1/2026 12:23 PM	When the file was first created on this volume
Last Accessed	4/1/2026	Last date the file was opened (FAT32 stores date only — no time component)
Modified	3/10/2025 4:37 PM	Last time the file content was changed

Part 3E - FAT Entries and Cluster Chain

The File Allocation Table (FAT) is a data structure that maps the cluster chain for each file on the volume. Each FAT32 entry is 4 bytes in size. Starting from cluster 5, the FAT entries were examined to identify all clusters allocated to Screenshot (1).png.

The file size is 218,973 bytes. With a cluster size of 4,096 bytes:

- $218,973 \div 4,096 = \sim 54$ clusters
- The file occupies clusters 5 through to approximately cluster 58
- The final cluster contains the end of chain marker 0x0FFFFFFF

Each FAT entry either points to the next cluster in the chain or contains the end of chain marker, allowing the operating system to reconstruct the complete file from non-contiguous clusters if necessary.

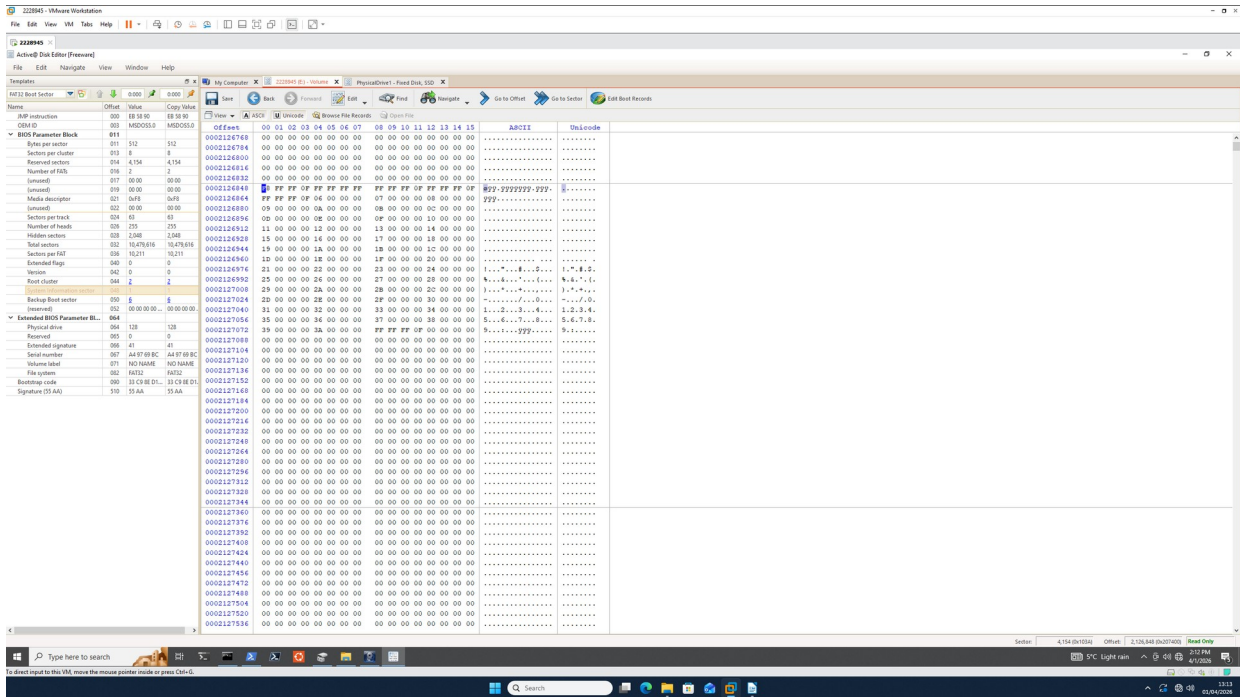


Figure 21: Active Disk Editor showing FAT1 (File Allocation Table) beginning at sector 4,154 (offset 0x2126848). The reserved clusters 0 and 1 show FF FF FF FF and FF FF FF 0F respectively, followed by the root directory cluster 2 showing end of chain marker 0x0FFFFFFF, and the sequential cluster chain entries for Screenshot (1).png beginning at cluster 5

Part 3F - File Deletion

The file Screenshot (1).png was deleted from the FAT32 volume using Shift+Delete to bypass the Recycle Bin. Active Disk Editor was then refreshed to examine the changes to the directory entry.

Field	Before Deletion	After Deletion
First byte of filename	0x53 (S)	0xE5 (deletion marker)
FAT chain entries	Cluster chain intact	Zeroed out (marked free)
File size in directory	218,973 bytes	0
Actual file data on disk	Present	Still present until overwritten

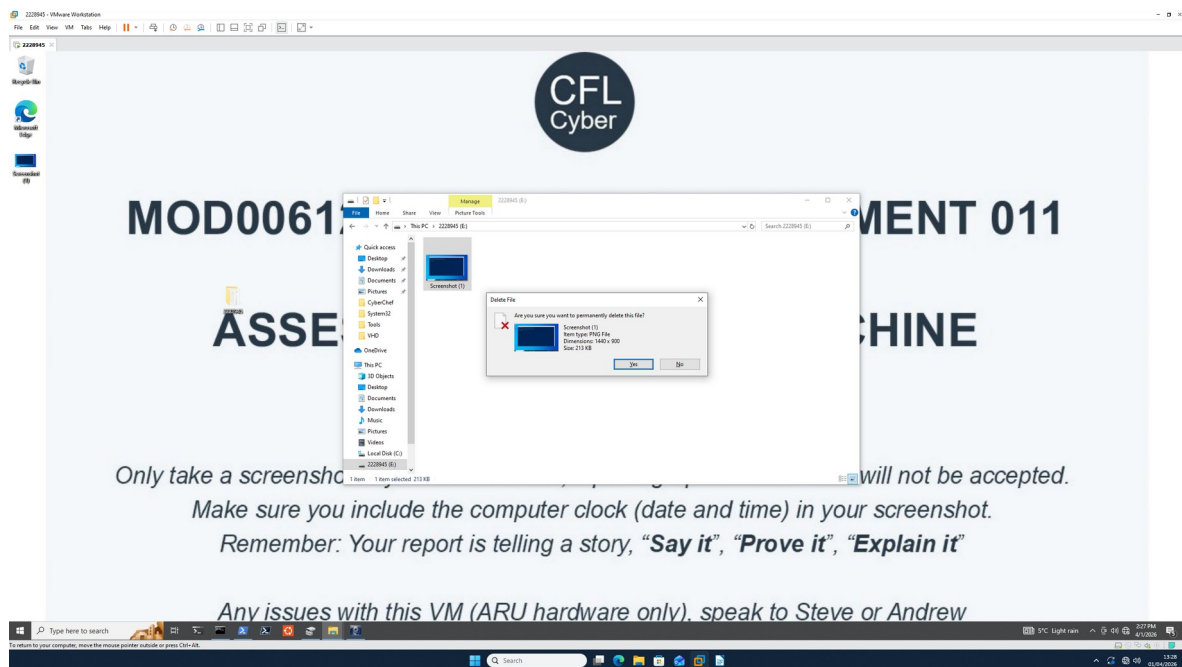


Figure 22: Permanent Deleting the screenshot from the drive

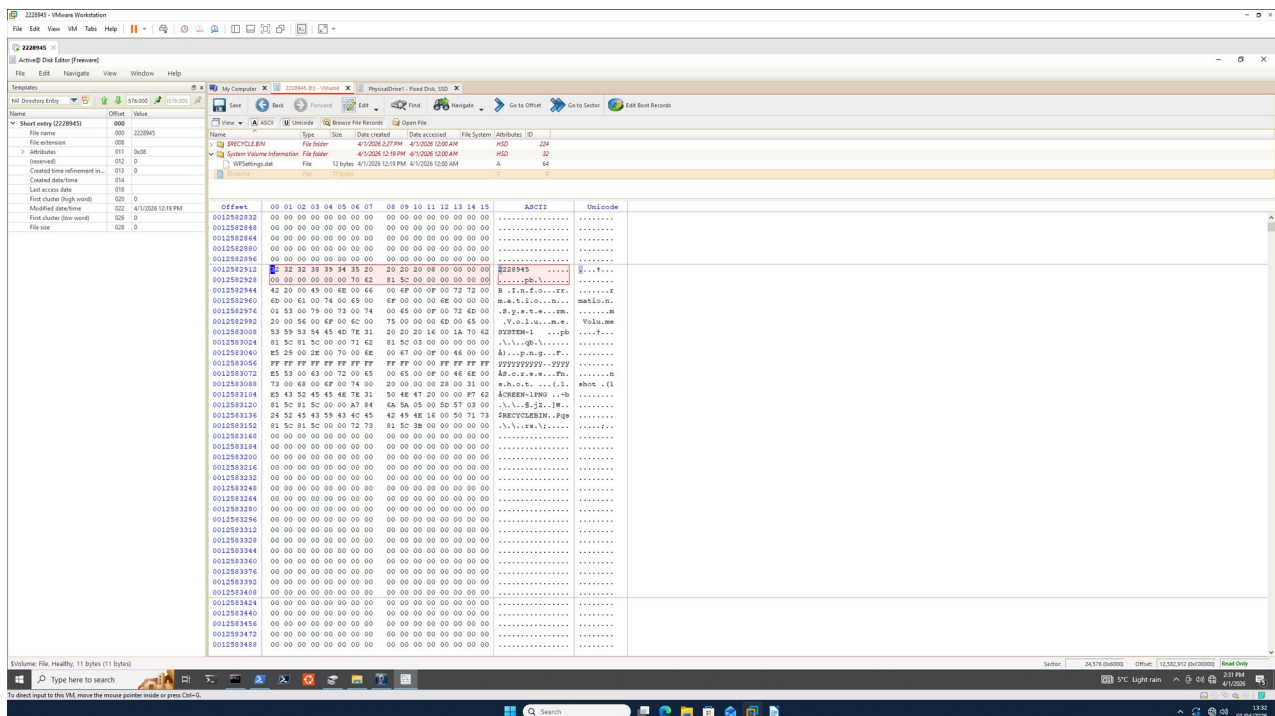


Figure 23: Active Disk Editor showing directory entry after deletion of Screenshot (1).png with 0xE5 deletion marker replacing the first byte of the filename

This is forensically significant as it means deleted FAT32 files can often be recovered using forensic tools, provided the clusters have not been overwritten. This is why secure deletion tools exist that overwrite cluster data multiple times to prevent recovery.

The image shows a Windows 10 desktop environment. In the foreground, a VMware Workstation window is open, displaying a Kali Linux virtual machine. The Kali Linux desktop is visible, showing a terminal window with a file search results. The terminal output lists various files and folders, including 'Long entry (Long)', 'Short entry (Short)', and 'Long entry (Long)'. The results are organized in a table with columns for file name, size, and timestamp. The background shows a Windows 10 taskbar with several open applications, including a web browser and a file explorer. The system clock in the bottom right corner indicates the date and time as 4/20/2024, 12:00 PM.

Figure 24: Active Disk Editor showing FAT entries zeroed out after deletion confirming clusters marked as free while actual file data remains on disk

Task 4 – NTFS Volume Creation and Forensic Imaging

4.1 Introduction to Task

Task 4 required the examiner to create a 5GB Virtual Hard Disk (VHD) formatted with the NT File System (NTFS) using a GUID Partition Table (GPT) partition scheme. A new directory named 'BOB' was created on the volume, and a file exceeding 200KB was copied from the SysWOW64 directory using a built-in console tool. The file properties were recorded, including name, timestamps, logical and physical size, location, and attributes. A selective forensic image of the file and BOB directory was then acquired using FTK Imager, with hash verification used to prove the integrity of the forensic image. The task also required an explanation of post-acquisition actions and a comparison between a logical volume copy and a selective file copy.

4.2 Methods and tools

Tool	Purpose
Disk Management (built-in)	VHD creation, GPT initialisation, NTFS formatting
Windows PowerShell	File properties, directory listing, size verification
Get-WmiObject CIM_DataFile	Logical size, timestamps, file metadata
Get-Item / Get-ChildItem	File attributes, full path, recursive directory listing
FTK Imager (v4.3.0.18)	Selective forensic image acquisition and hash verification

Step 1: A 5GB fixed-size VHD was created using Disk Management and initialised using a GPT partition scheme. The volume was formatted with NTFS and assigned the volume label 'NTFS2228945'. GPT was selected as it is the modern partition scheme required for NTFS volumes and supports drives over 2TB with improved redundancy over MBR.

Step 2: A new directory named 'BOB' was created on the NTFS volume. The file AcGenral.dll was identified in the C:\Windows\SysWOW64 directory as it exceeded the 200KB requirement at 2,407,424 bytes. The file was copied to the BOB directory using the PowerShell Copy-Item cmdlet, which is a built-in console tool.

Step 3: PowerShell cmdlets Get-Item and Get-WmiObject CIM_DataFile were used to retrieve all required file properties including name, timestamps, logical size, physical size, location and attributes.

Step 4: FTK Imager was used to perform a selective forensic acquisition of the AcGenral.dll file and the BOB directory from the NTFS volume. The image was saved in E01 format and hash verified upon completion.

4.3 Relevant Data Identified

Part 4A - NTFS Volume Creation

A 5GB VHD was created using Disk Management and initialised with a GPT partition scheme. The volume was formatted with NTFS and labelled 'NTFS2228945'. The GPT partition scheme was selected as it is the modern industry standard for NTFS volumes, storing partition data in multiple locations on the disk for redundancy, unlike MBR which stores it only at sector 0.

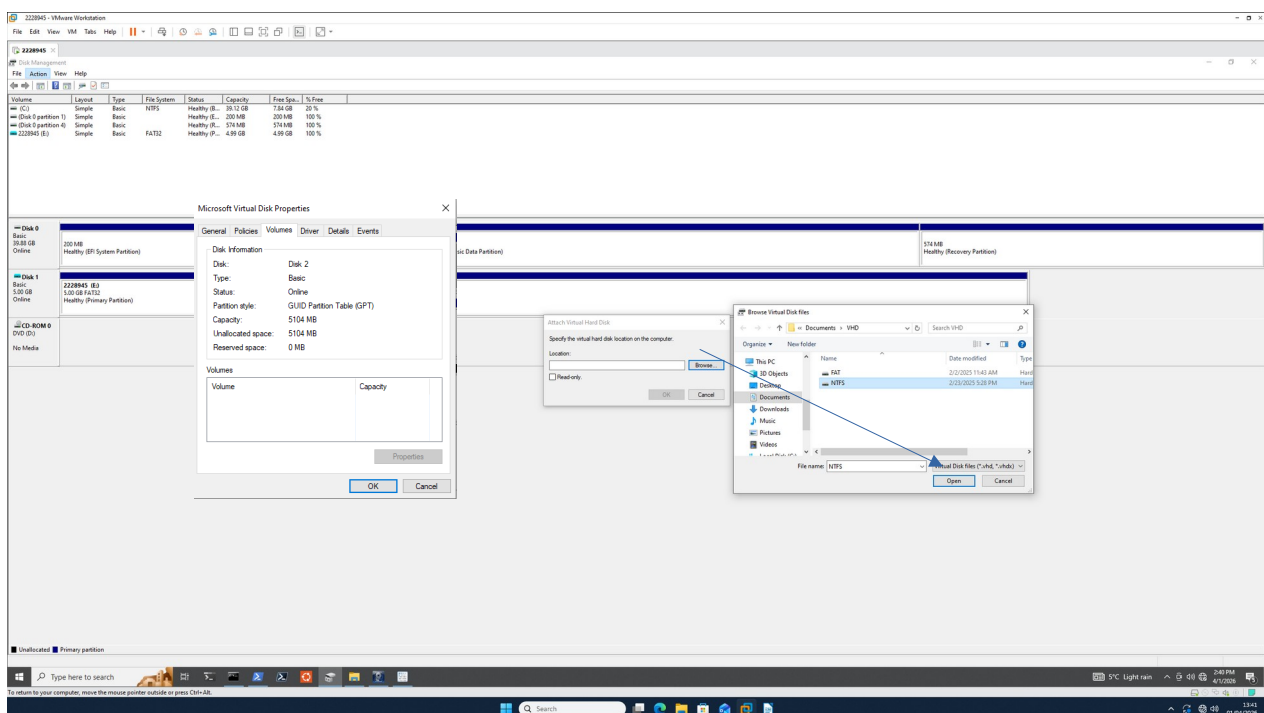


Figure 25: Disk Management showing VHD creation dialog and GPT initialisation selected



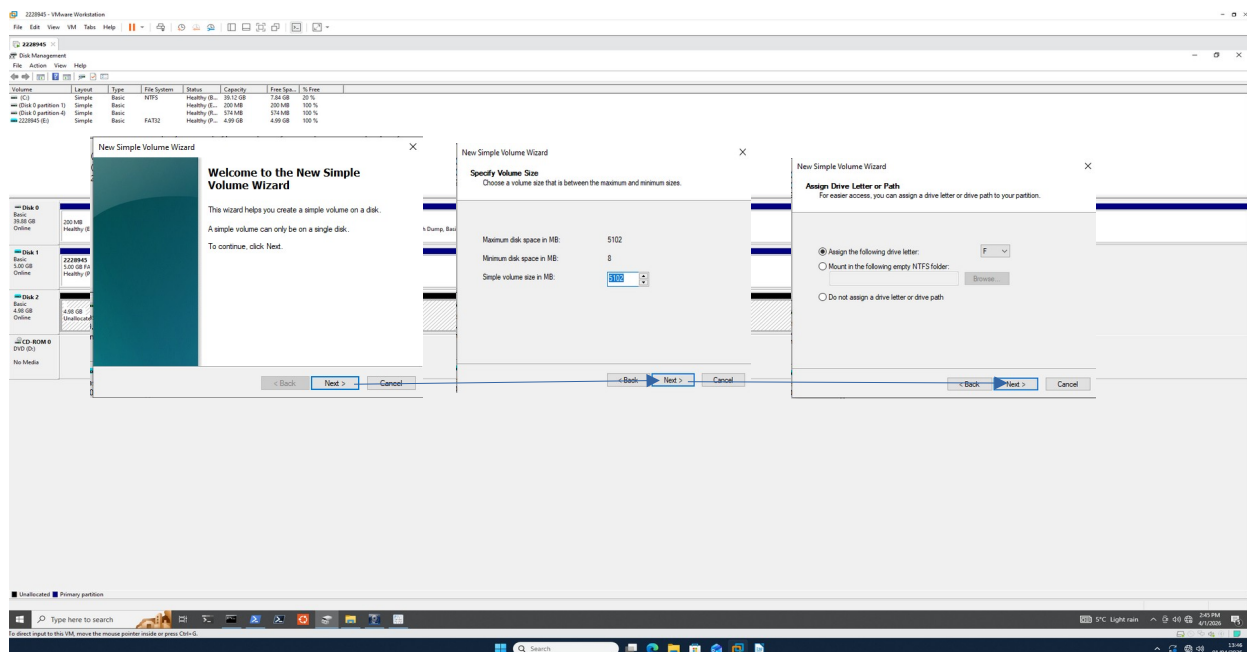


Figure 26: New Simple Volume Wizard showing volume size (5102MB), drive letter assignment and NTFS format with volume label NTFS2228945

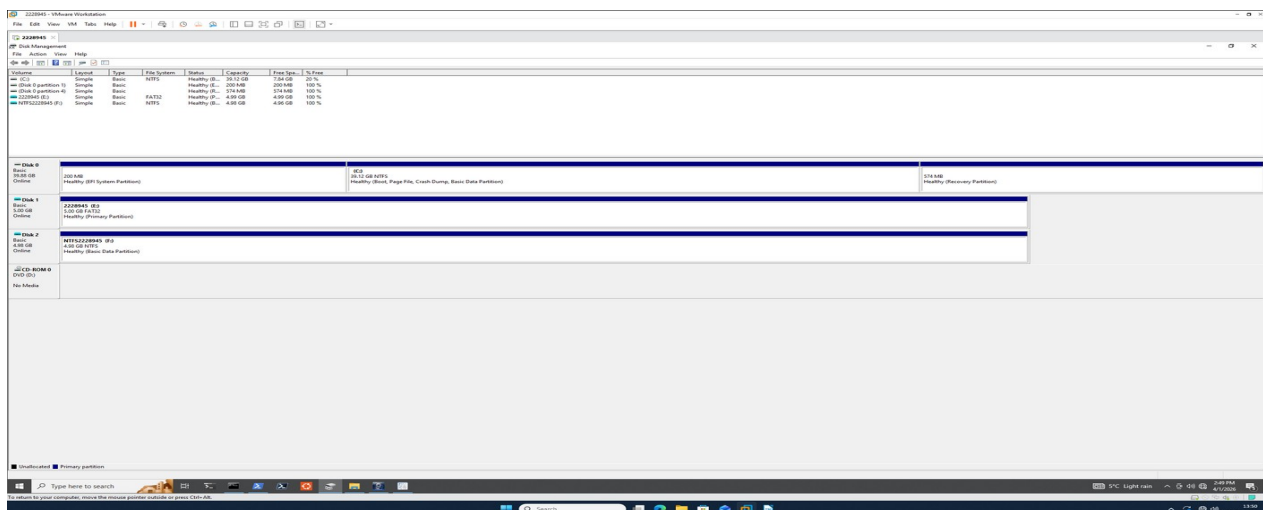


Figure 27: Disk Management showing completed NTFS volume on Disk 2 with GPT partition scheme

Part 4B - File Properties

The file AcGenral.dll was copied from C:\Windows\SysWOW64 to F:\BOB using the PowerShell Copy-Item cmdlet. The following properties were recorded:

Property	Value
Name	AcGenral.dll
Full Path	F:\BOB\AcGenral.dll
Logical Size	2,407,424 bytes

Physical Size	2,408,448 bytes (size on disk)
Creation Time	4/1/2026 2:56:46 PM
Last Accessed	4/1/2026 2:59:06 PM
Last Modified	11/27/2024 12:00:24 PM
Attributes	Archive
Location	F:\BOB

The difference between logical size (2,407,424 bytes) and physical size (2,408,448 bytes) is due to NTFS cluster allocation. Files are allocated in whole clusters, meaning the final cluster may not be fully used, resulting in slack space of 1,024 bytes.

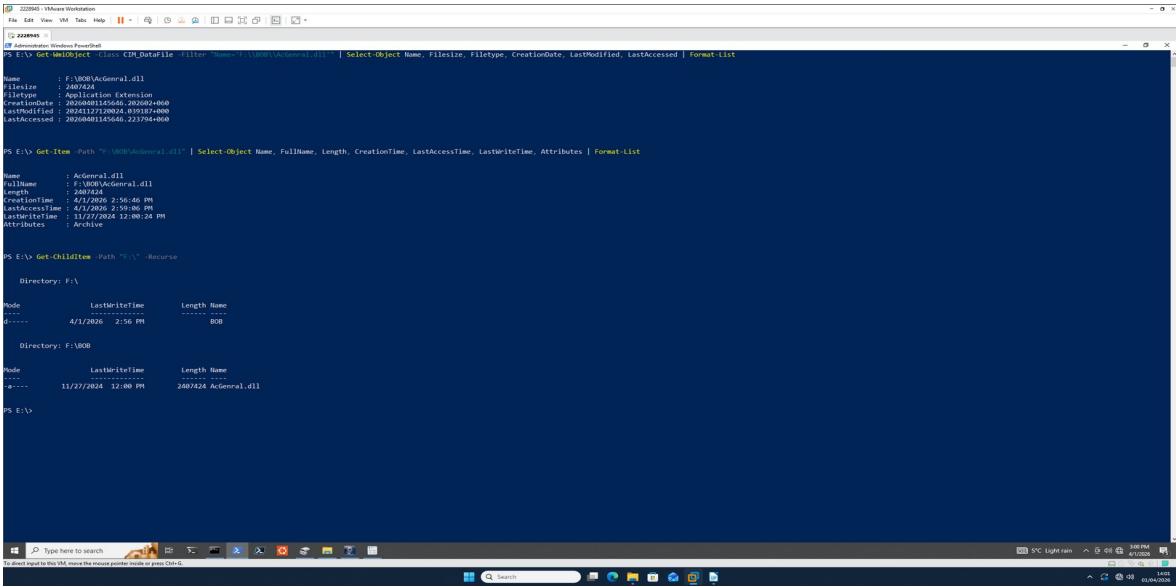


Figure 28: Using Powershell to gather information such as ogical size, filetype , timestamps , full path, length, creation time, last access time, last write time and attributes for AcGenral.dll as well as -Recurse output showing BOB directory and AcGenral.dll with mode flags and timestamps

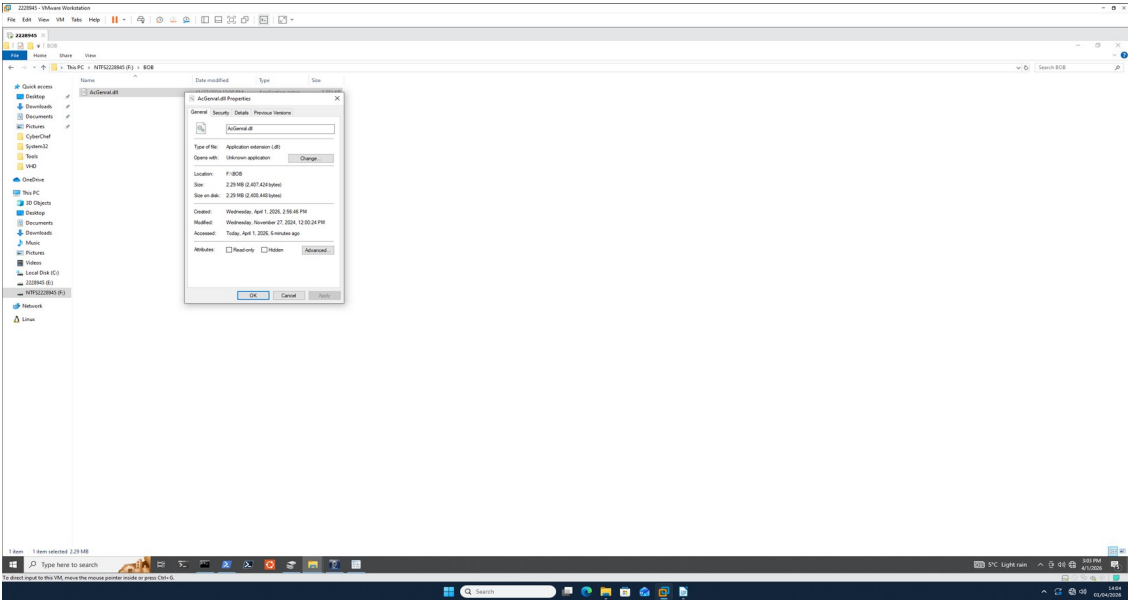


Figure 29: Windows Explorer Properties window for AcGenral.dll showing size (2,407,424 bytes) and size on disk (2,408,448 bytes)

Part 4C – Selective Forensic Image Acquisition

A selective forensic image of AcGenral.dll and the BOB directory was acquired using FTK Imager in E01 format. The following case details were recorded:

Field	Value
Case Number	2228945
Evidence Number	1
Unique Description	Task4/BOB directory
Examiner	Steve
Image Format	E01

Following acquisition, FTK Imager automatically verified the image by recomputing hash values:

Hash Algorithm	Computed Hash	Report Hash
MD5	1b8b9a4fe94d0b111779c4f75a0d7066	1b8b9a4fe94d0b111779c4f75a0d7066
SHA1	402ed7648bb64424a62798eae747ab03b96ff	402ed7648bb64424a62798eae747ab03b96ff

The matching hash values confirm the forensic image is an identical copy of the original data and has not been altered during acquisition.

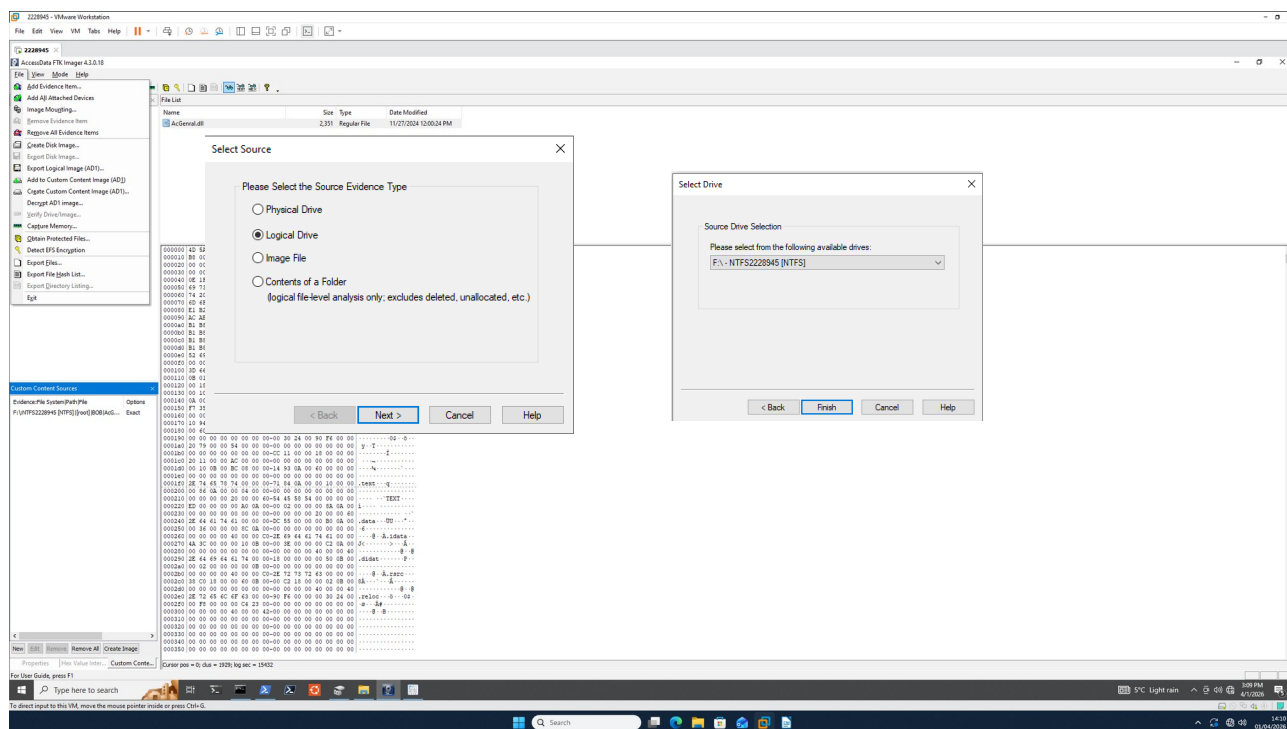


Figure 30: FTK Imager Select Source dialog showing Logical Drive selected and NTFS2228945 volume as source

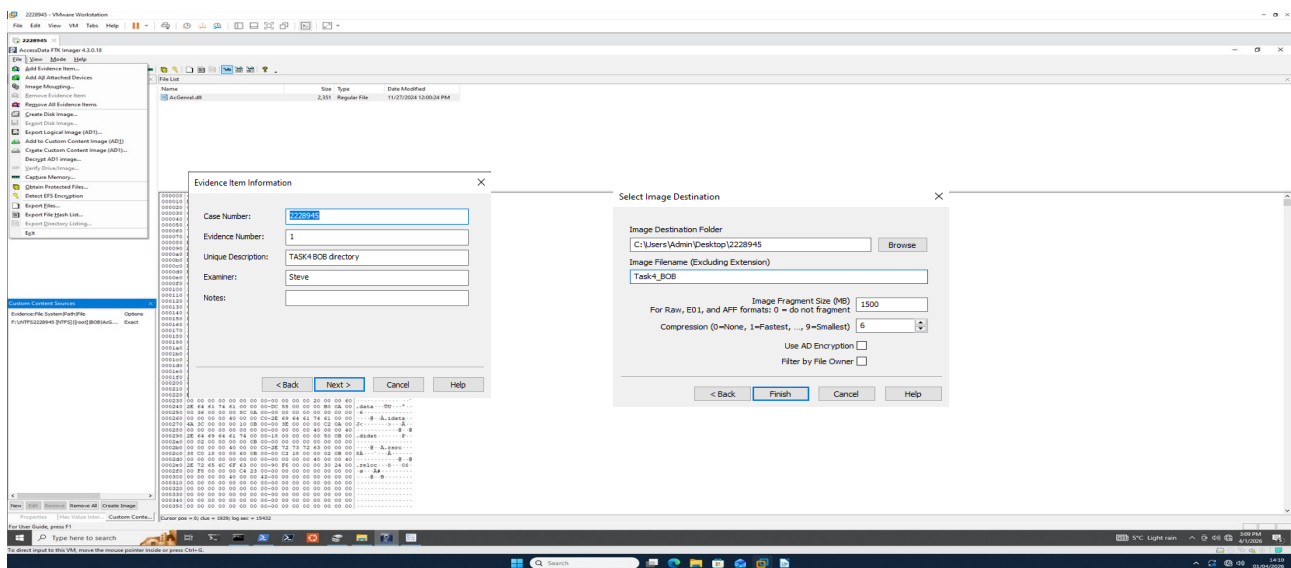


Figure 31: FTK Imager Evidence Item Information dialog showing case number 2228945, evidence number 1, description Task4/BOB directory and examiner Steve

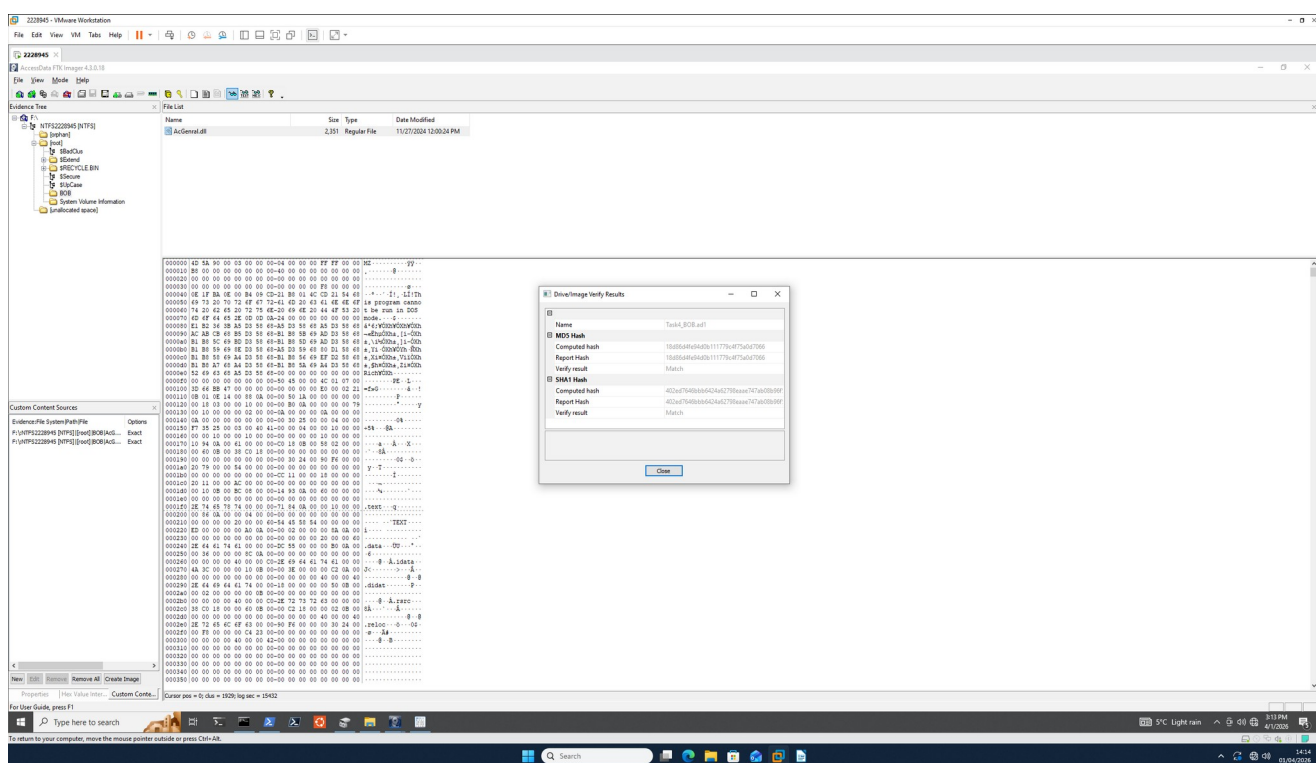


Figure 32: TK Imager Drive/Image Verify Results showing MD5 and SHA1 computed hashes matching report hashes with Match result confirmed for image Task4_BOB.e01

Part 4D - Post-Acquisition Actions

Prior to commencing analysis of the forensic image, the following actions would be taken. Firstly, the hash values of the image would be independently verified using PowerShell's Get-FileHash cmdlet to confirm the image had not been altered since acquisition. Secondly, the original evidence would be stored securely and write-protected to prevent any modification. Analysis would be conducted on a working copy of the forensic image rather than the original, preserving the integrity of the original evidence. The forensic image would be mounted as read-only within FTK Imager before any examination begins. Finally, a chain of custody document would be completed recording who acquired the image, when, and under what conditions. These actions adhere to ACPO Principle 2, which requires that any person accessing digital evidence must be competent to do so and able to explain their actions (ACPO, 2012).

Part 4E - Logical Copy vs Selective Copy

A logical volume copy captures the entire contents of a volume including all files, directories, deleted file remnants, unallocated clusters, slack space and file system metadata. This provides the most forensically complete picture of the volume and allows recovery of deleted files and other artefacts that would otherwise be inaccessible.

A selective copy, such as the one acquired in this task, captures only the specified file and directory in this case AcGenral.dll and the BOB directory. While the selective image still contains a verified hash proving its integrity, it does not preserve unallocated space, slack space, or deleted data from the wider volume. This means that evidence which may exist outside the selected items cannot be recovered from a selective image alone. A selective copy is therefore faster and more targeted but is forensically less complete than a full logical or physical volume image (Casey, 2011).

TASK 5

5.1 Introduction to Task

Task 5 required the examiner to analyse the NTFS metadata associated with the BOB directory and the AcGenral.dll file copied to it in Task 4. This included identifying the MFT Record IDs for the file, directory and parent directory, together with an explanation of parenting. The task also required an explanation of MAC time anomalies, how NTFS differentiates between files and directories, the location of the Volume Serial Number on an NTFS file system, any anomalies between physical and logical data, and a demonstration of running and terminated processes on the virtual machine.

Tool	Purpose
Windows PowerShell	MFT Record ID retrieval, MAC times, VSN, process listing
fsutil.exe (built-in)	MFT Record ID queries via queryFileID
Get-Item / Get-ChildItem	File and directory timestamp retrieval
Active Disk Editor	Physical examination of \$Boot sector, MFT records
Get-Process / Get-WmiObject	Running and historical process identification
Windows Prefetch	Evidence of terminated/orphaned processes

Step 1: PowerShell's fsutil file queryFileID cmdlet was used to retrieve the MFT Record IDs for AcGenral.dll, the BOB directory and the root directory of the NTFS volume.

Step 2: Get-Item was used to retrieve all MAC timestamps for both AcGenral.dll and the BOB directory, and any anomalies were identified and explained.

Step 3: Active Disk Editor was used to examine the \$Boot sector of the NTFS volume to physically locate the Volume Serial Number and demonstrate the byte-order anomaly between physical and logical representations.

Step 4: Active Disk Editor was used to examine MFT records for both the file and directory to demonstrate how NTFS differentiates between the two.

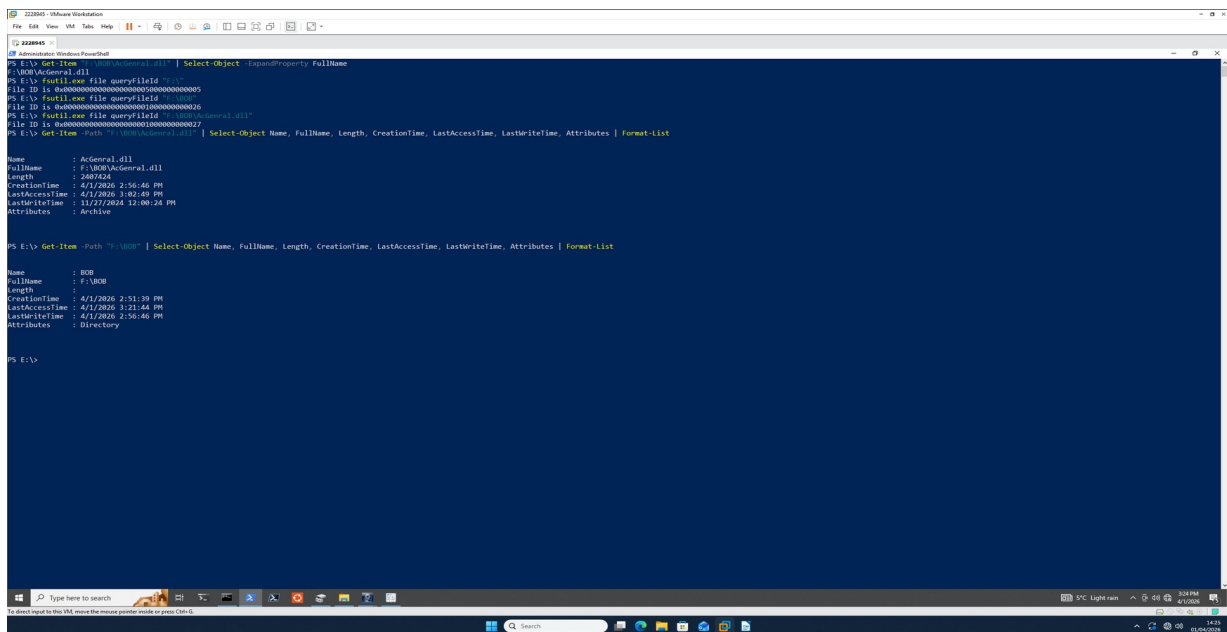
Step 5: PowerShell's Get-Process and Get-WmiObject Win32_Process cmdlets were used to list running processes, and the Windows Prefetch directory was examined to identify evidence of terminated processes.

5.3 Relevant Data Identified

Part 5A - MFT Record IDs and Parenting

The MFT (Master File Table) is a core NTFS structure in which every file and directory on the volume has a corresponding record. Each record is identified by a unique MFT Record ID. The following IDs were retrieved using the fsutil file queryFileID cmdlet

Item	Full Path	MFT Record ID
Root Directory (Parent)	F:\	5
BOB Directory	F:\BOB	38
AcGenral.dll (File)	F:\BOB\AcGenral.dll	39



Parenting in NTFS works through the \$FILE_NAME attribute stored within each MFT record. This attribute contains the MFT Record ID of the parent directory. In this case, AcGenral.dll (MFT 39) contains a parent reference pointing to BOB (MFT 38), and BOB (MFT 38) contains a parent reference pointing to the root directory (MFT 5). This chain of parent references allows the operating system to reconstruct the full file path from any MFT record without traversing the entire directory tree.

The following MAC timestamps were identified for AcGenral.dll and the BOB directory:
AcGenral.dll:

BOB Directory:

MAC Time Anomaly - AcGenral.dll:

A significant forensic anomaly is present in the timestamps of AcGenral.dll. The Last Write Time (Modified) of 11/27/2024 is approximately 16 months earlier than the Creation Time of 4/1/2026. Under normal circumstances, a file cannot be modified before it was created. This anomaly occurs because when a file is copied using the PowerShell Copy-Item cmdlet, NTFS preserves the original Last Write Time from the source file in SysWOW64, while the Creation Time is set to the time the file was copied to the BOB directory. This is a well-documented forensic indicator that a file has been copied rather than created natively on the volume. An examiner encountering this anomaly should investigate the source of the file, as the Modified timestamp reflects the original file's last modification date on the source system (Casey, 2011).

BOB Directory - No Anomaly:

The BOB directory timestamps show no anomaly. The Creation Time (2:51:39 PM) predates the Last Write Time (2:56:46 PM), which is consistent with the directory being created first and then modified when AcGenral.dll was copied into it.

Part 5C – How NTFS Differentiates Files and Directories

NTFS differentiates between files and directories through the MFT record header flags and the attributes stored within each record. In the MFT record header, a flags field indicates the type of entry. A value of 0x0000 indicates a regular file, while a value of 0x0002 indicates a directory.

Beyond the header flags, the attributes stored within the MFT record also differ between files and directories. A file record contains a \$DATA attribute which holds the actual file content, either inline within the MFT record for small files or pointing to data runs on the volume for larger files. A directory record does not contain a \$DATA attribute. Instead, it contains an \$INDEX_ROOT attribute and, for large directories, an \$INDEX_ALLOCATION attribute. These index attributes store the directory entries listing the files and subdirectories contained within that directory, organised as a B-tree structure for efficient searching.

[Figure to be inserted: Active Disk Editor showing MFT record for AcGenral.dll with \$DATA attribute, and MFT record for BOB directory showing \$INDEX_ROOT attribute and directory flag 0x0002]

Part 5D - NTFS Volume Serial Number and Physical/Logical Anomaly

The Volume Serial Number (VSN) for the NTFS volume was retrieved logically using PowerShell:

VolumeSerialNumber: **203B0C3A**

The VSN was then physically located within the \$Boot sector of the NTFS volume using Active Disk Editor. On an NTFS volume the VSN is stored within the BIOS Parameter Block of the \$Boot sector at offset 0x48, as an 8-byte value stored in little-endian format.

The VSN was then physically located within the \$Boot sector of the NTFS volume using Active Disk Editor. On an NTFS volume the VSN is stored within the BIOS Parameter Block of the \$Boot sector at offset 0x48, as an 8-byte value stored in little-endian format.

Source	Value
PowerShell - logical	203B0C3A
Active Disk Editor - physical (offset 0x48)	3A 9C 5B 26 C6 5B 26 60

This represents the same anomaly observed in Task 2 with the FAT32 volume. The operating system reverses the byte order when presenting the VSN logically to the user, while the raw physical bytes on disk are stored in little-endian format. This is forensically significant as an examiner working directly with raw disk data must account for byte-order differences when comparing physical and logical values.

The \$Boot sector also confirmed the following NTFS volume parameters:

Parameter	Value
OEM ID	NTFS
Bytes per Sector	512
Sectors per Cluster	8
Total Sectors	10,448,895
\$MFT Cluster Number	262,144
\$MFTMirr Cluster Number	2

```

Administrator: Windows PowerShell
PS E:\> Get-WmiObject -Namespace 'Wbem\\' -Class 'Win32_LogicalDisk' -Filter 'MediaType=Fixed' | Select-Object VolumeSerialNumber | Format-List

VolumeSerialNumber : 203B0C3A

PS E:\>

```

Figure 34: PowerShell Get-WmiObject output showing logical VSN 203B0C3A

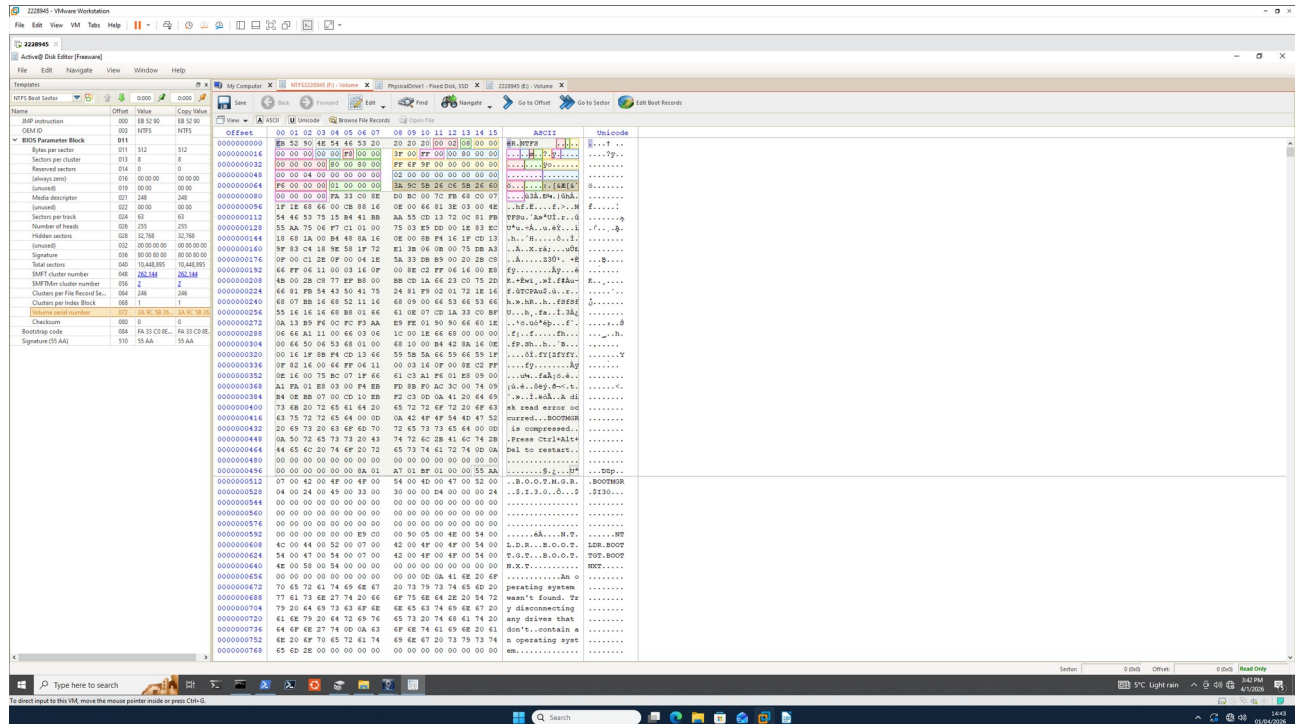


Figure 35: Active Disk Editor showing NTFS \$Boot sector BIOS Parameter Block with Volume Serial Number highlighted at offset 0x48 showing physical value 3A 9C 5B 26,

Part 5E - Running and Terminated Processes

Running processes on the virtual machine were identified using two PowerShell cmdlets. Get-WmiObject Win32_Process was used to retrieve detailed process metadata including Name, ProcessID and CreationDate, while Get-Process was used to retrieve Name, ID and StartTime for all active processes.

The following processes were identified as running at the time of examination:

Process Name	PID	Start Time
cmd.exe	4428	4/1/2026 11:31:14 AM
csrss	530	4/1/2026 11:17:37 AM
dllhost	3968	4/1/2026 11:25:50 AM
conhost	4544	4/1/2026 12:40:13 PM
DiskEditor	1764	4/1/2026 3:38:53 PM
Registry	4	4/1/2026 11:17:51 AM
svchost	344	4/1/2026 11:17:51 AM

```
PS E:\> Get-WmiObject Win32_Process | Select-Object Name, ID, StartTime | Format-List

Name       : ADIso
ID          : 1264
StartTime  : 4/1/2026 12:48:13 PM

Name       : AggregatorHost
ID          : 6480
StartTime  : 4/1/2026 11:17:53 AM

Name       : ApplicationFrameHost
ID          : 6432
StartTime  : 4/1/2026 11:19:19 AM

Name       : cmd
ID          : 7420
StartTime  : 4/1/2026 11:31:14 AM

Name       : conhost
ID          : 3752
StartTime  : 4/1/2026 11:24:23 AM

Name       : conhost
ID          : 6544
StartTime  : 4/1/2026 12:48:13 PM

Name       : conhost
ID          : 9004
StartTime  : 4/1/2026 11:31:14 AM

Name       : csrss
ID          : 436
StartTime  : 4/1/2026 11:17:37 AM

Name       : csrss
ID          : 520
StartTime  : 4/1/2026 11:17:37 AM

Name       : ctfmon
ID          : 5132
StartTime  : 4/1/2026 11:18:05 AM

Name       : DiskEditor
ID          : 1784
StartTime  : 4/1/2026 3:30:53 PM

Name       : dlhost
ID          : 3864
StartTime  : 4/1/2026 11:29:58 AM

Name       : dlhost
ID          : 3868
StartTime  : 4/1/2026 11:17:48 AM

Name       : dlhost
ID          : 4064
StartTime  : 4/1/2026 11:17:49 AM

Name       : dsm
```

Figure 36: PowerShell Get-WmiObject Win32_Process output showing running processes with Name, ProcessID and CreationDate

```
PS E:\> Get-Process | Select-Object Name, ProcessID, ParentProcessID, CreationDate | Format-List

Name       : System Idle Process
ProcessID  : 0
ParentProcessID : 0
CreationDate : 202604011173.927409+000

Name       : System
ProcessID  : 4
ParentProcessID : 0
CreationDate : 202604011173.927409+000

Name       : Registry
ProcessID  : 92
ParentProcessID : 4
CreationDate : 202604011173.882822+000

Name       : smss.exe
ProcessID  : 388
ParentProcessID : 4
CreationDate : 202604011173.863314+000

Name       : csrss.exe
ProcessID  : 436
ParentProcessID : 436
CreationDate : 202604011173.277908+000

Name       : wininit.exe
ProcessID  : 512
ParentProcessID : 436
CreationDate : 202604011173.740019+000

Name       : csrss.exe
ProcessID  : 520
ParentProcessID : 504
CreationDate : 202604011173.752001+000

Name       : winlogon.exe
ProcessID  : 388
ParentProcessID : 584
CreationDate : 202604011173.839517+000

Name       : services.exe
ProcessID  : 612
ParentProcessID : 512
CreationDate : 202604011173.885085+000

Name       : lsass.exe
ProcessID  : 612
ParentProcessID : 512
CreationDate : 202604011173.136217+000

Name       : vmtoolsd.exe
ProcessID  : 748
ParentProcessID : 612
CreationDate : 202604011173.973772+000

Name       : vmtoolsd.exe
ProcessID  : 788
ParentProcessID : 788
```

Figure 37: PowerShell Get-Process output showing running processes with Name, ID and StartTime

Terminated and Orphaned Processes:

Processes that have terminated or become orphaned can no longer be identified through live process listing tools such as Get-Process, as these only display currently active processes. There are two primary methods for identifying terminated or orphaned processes.

The first method is examination of Windows Prefetch files, located in C:\Windows\Prefetch. Prefetch files are created by the Windows Superfetch service the first time an application is executed, and are updated on subsequent executions. Each Prefetch file records the executable name and the last execution time, meaning that even after a process has terminated, evidence of its

execution persists in the Prefetch directory. By examining the LastWriteTime of Prefetch files, an examiner can identify applications that have previously run on the system.

[Figure to be inserted: PowerShell Get-ChildItem C:\Windows\Prefetch output showing Prefetch files with LastWriteTime values confirming previously executed processes]

The second method is analysis of the physical memory dump acquired in Task 1. Tools such as Volatility Framework can analyse the raw memory image to identify process structures (EPROCESS blocks) that remain in memory even after a process has terminated. This allows recovery of orphaned or terminated process metadata including process name, PID, parent PID and execution timestamps, which would not be visible through live operating system tools (Nelson et al., 2008).

Task-6

6.1 Introduction to Task

Task 6 required collection of live system evidence from the virtual machine using console utilities, covering network configuration, active TCP and UDP connections, firewall rules, Remote Desktop Protocol (RDP) status, and network shares. Live data is volatile and must be collected in accordance with the order of volatility (Brezinski and Killalea, 2002) and ACPO Principle 3, which requires an audit trail of all processes applied to digital evidence (ACPO, 2012; ISO, 2012).

6.2 Method and Tools

Tool	Purpose
ipconfig /all	Network configuration and MAC address
Get-NetAdapter	Adapter name, MAC and status (verification)
netstat -ano	Active TCP/UDP connections with owning PIDs
Get-NetFirewallRule	Firewall rules and exceptions
Get-ItemProperty (Registry)	RDP enabled/disabled status and port number
Get-SmbShare / net share	Network share identification

All commands were executed from an elevated PowerShell session and captured as full-screen screenshots including the virtual machine clock for evidential timestamping.

6.3 Relevant Data Identified

```
2228945 - VMware Workstation
Administrator Windows PowerShell
PS E:\> ipconfig /all

Windows IP Configuration

Host Name . . . . . : DESKTOP-MD4QGGB
Primary Dns Suffix . . . . . : localdomain
Node Type . . . . . : Hybrid
IP Routing Enabled. . . . . : No
WINS Proxy Enabled. . . . . : No

Ethernet adapter Ethernet0:

Media State . . . . . : Media disconnected
Connection-specific DNS Suffix . : localdomain
Description . . . . . : Intel(R) 82574L Gigabit Network Connection
Physical Address. . . . . : 00-0C-29-E1-CB-F7
DHCP Enabled. . . . . : Yes
Autoconfiguration Enabled . . . . : Yes

PS E:\> NETSTAT.EXE -ano

Active Connections

Proto Local Address           Foreign Address         State       PID
TCP    0.0.0.0:135              0.0.0.0:0               LISTENING   876
TCP    0.0.0.0:445              0.0.0.0:0               LISTENING   4
TCP    0.0.0.0:5040             0.0.0.0:0               LISTENING   5224
TCP    0.0.0.0:5040             0.0.0.0:0               LISTENING   5404
TCP    0.0.0.0:7680             0.0.0.0:0               LISTENING   672
TCP    0.0.0.0:49664            0.0.0.0:0               LISTENING   512
TCP    0.0.0.0:49665            0.0.0.0:0               LISTENING   896
TCP    0.0.0.0:49666            0.0.0.0:0               LISTENING   2560
TCP    0.0.0.0:49667            0.0.0.0:0               LISTENING   2544
TCP    0.0.0.0:49668            0.0.0.0:0               LISTENING   652
TCP    0.0.0.0:49669            0.0.0.0:0               LISTENING   876
TCP    [::]:135                 [::]:0                  LISTENING   4
TCP    [::]:445                 [::]:0                  LISTENING   5404
TCP    [::]:5040                [::]:0                  LISTENING   672
TCP    [::]:49664               [::]:0                  LISTENING   512
TCP    [::]:49665               [::]:0                  LISTENING   896
TCP    [::]:49666               [::]:0                  LISTENING   2560
TCP    [::]:49667               [::]:0                  LISTENING   2544
TCP    [::]:49668               [::]:0                  LISTENING   652
TCP    [::]:49669               [::]:0                  LISTENING   876
UDP    0.0.0.0:123              *:*                     7788
UDP    0.0.0.0:5040             *:*                     5224
UDP    127.0.0.1:1900           *:*                     5680
UDP    127.0.0.1:49664         *:*                     3240
UDP    127.0.0.1:49665         *:*                     5680
UDP    [::]:123                 *:*                     7788
UDP    [::]:1900                *:*                     5680
UDP    [::]:49665               *:*                     5680
```

Figure 38: ipconfig /all output showing host name, adapter details and media disconnected state

Part 6A – Network Configuration

Network configuration was obtained using ipconfig /all (Figure 40) and independently verified using Get-NetAdapter (Figure 41). Both commands returned the same adapter identity:

Property	Value
Host Name	DESKTOP-MD4QGGB
Adapter	Ethernet0 Intel(R) 82574L Gigabit Network Connection
MAC Address	00-0C-29-E1-CB-F7
Media State	Disconnected
DNS Suffix	localdomain
DHCP Enabled	Yes
IP Routing Enabled	No

The MAC address prefix 00-0C-29 is the Organizationally Unique Identifier assigned to VMware Inc., confirming the adapter is virtualised (IEEE, 2024). The adapter reported a disconnected

state, indicating no active external network connectivity at the time of examination. The use of two independent commands returning identical results demonstrates evidential consistency.

Part 6B – Network Connections Including TCP

Active connections were identified using `netstat -ano`, which lists TCP and UDP endpoints with their owning Process IDs (Figure 40). All TCP connections observed were in the LISTENING state with no ESTABLISHED outbound connections, consistent with the disconnected adapter. Ports of forensic interest included:

Port / Service	Purpose
TCP 135 (PID 876)	Microsoft RPC Endpoint Mapper
TCP 445 (PID 4)	SMB / Microsoft Directory Services (System process)
TCP 49664 - 49669	Windows dynamic ephemeral ports for system services
TCP 7680 (PID 5404)	Delivery Optimization (Windows Update peer-to-peer)
UDP 123 (PID 7788)	Windows Time Service (NTP)
UDP 5050	Local service binding
UDP 127.0.0.1 loopback	Inter-process communication

Listening services on ports 135 and 445 are of particular forensic interest as they are common vectors for lateral movement and remote code execution. The PID column allows each listening port to be correlated with the process listings obtained in Task 5 to identify the responsible executable.

```

PS E:\> Get-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Control\Terminal\Server" | Select-Object fDenyConnections | Format-List
fDenyConnections : 1

PS E:\> Get-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Control\Terminal\Server" | Select-Object PortNumber | Format-List
PortNumber :

PS E:\> Get-SmbShare | Format-List
Name           : ADMIN$
ScopeName      : *
Path           : C:\WINDOWS
Description    : Remote Admin
Name           : C$
ScopeName      : *
Path           : C:\
Description    : Default share
Name           : E$
ScopeName      : *
Path           : E:\
Description    : Default share
Name           : F$
ScopeName      : *
Path           : F:\
Description    : Default share
Name           : IPC$
ScopeName      : *
Path           :
Description    : Remote IPC

PS E:\> net share
Share name      Resource
-----
C$              C:\
E$              E:\
F$              F:\
IPC$            C:\WINDOWS
ADMIN$          Remote Admin
The command completed successfully.
PS E:\>

```

Figure 39: Figure 41: PowerShell Get-NetFirewallRule output showing representative firewall rules with Name, Enabled state, Direction and Action fields, including SNMPTRAP, WiFiDirect, DeliveryOptimization and PNRPMNRS rules

Part 6C – Firewall Rules and Exceptions

Firewall rules were enumerated using Get-NetFirewallRule with a selection of Name, Enabled, Direction and Action for readability (Figure 42). Representative rules observed included:

Rule Name	State
SNMPTRAP-In-UDP	Disabled, Inbound, Allow
WiFiDirect-KM-Driver-In-TCP	Enabled, Inbound, Allow
WiFiDirect-KM-Driver-Out-TCP	Enabled, Outbound, Allow
DeliveryOptimization-TCP-In	Enabled, Inbound, Allow
DeliveryOptimization-UDP-In	Enabled, Inbound, Allow
PNRPMNRS-PNRP-In-UDP	Disabled, Inbound, Allow

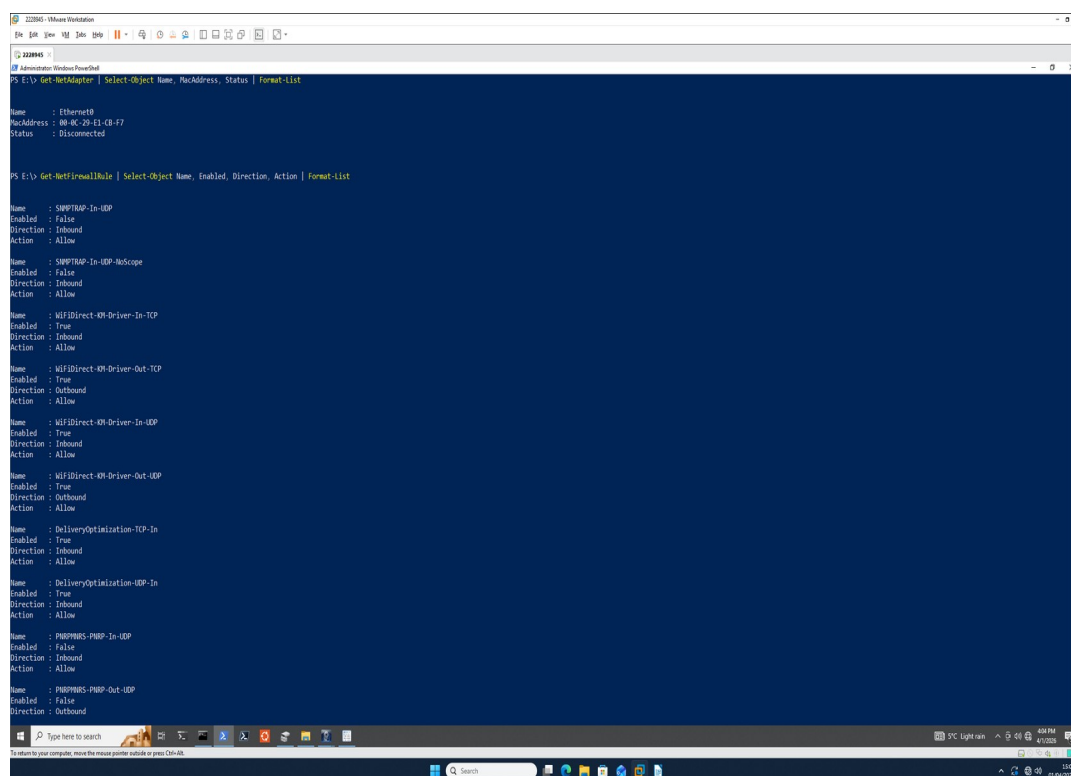


Figure 40: Figure 42: PowerShell Get-NetAdapter output showing Ethernet0 adapter with MAC address 00-0C-29-E1-CB-F7 and Disconnected status

Firewall enumeration is essential for understanding the system's attack surface and identifying any non-default or attacker-created rules. In a full investigation, rules allowing inbound traffic on non-standard ports, rules referencing uncommon executables, and rules created by non-system accounts would all warrant further investigation. The command netsh advfirewall show allprofiles would additionally confirm the default policies of each firewall profile (Domain, Private, Public).

Part 6D – RDP Remote Access Information and Status

RDP configuration was examined by querying the registry key HKLM:

\SYSTEM\CurrentControlSet\Control\Terminal Server\ using Get-ItemProperty (Figure 43):

Registry Value	Observed Value	Meaning
fDenyTSConnections	1	RDP connections DENIED (disabled)
PortNumber	(empty / default)	Default TCP 3389 if re-enabled

The fDenyTSConnections value is the primary toggle for RDP on a Windows host: a value of 1 denies incoming connections, while 0 enables the service. The empty PortNumber means the default port 3389 would be used if re-enabled.

Forensic Significance of the RDP Configuration

That RDP is disabled establishes the virtual machine was not exposing an interactive remote logon surface at the time of examination, reducing the likelihood of RDP-based intrusion such as credential brute-forcing or exploitation of legacy vulnerabilities (e.g. CVE-2019-0708 BlueKeep). However, an attacker may have disabled RDP after use to frustrate detection, so the current state

alone is not conclusive. A full investigation would additionally review the Security event log (Event IDs 4624 and 4625 with Logon Type 10) for historical RDP logon attempts, examine the registry key's LastWriteTime to determine when fDenyTSConnections was last modified, check the TermService service state and start type, and review any saved RDP credentials or .rdp files in the user profile for evidence of outbound RDP activity (Carvey, 2018).

Part 6E – Network Shares

Network shares were enumerated using Get-SmbShare and net share (Figure 43), with both tools returning identical results:

Share Name	Path / Description
ADMIN\$	C:\WINDOWS → Remote Admin
C\$	C:\ → Default share
E\$	E:\ → Default share (FAT32 from Task 2)
F\$	F:\ → Default share (NTFS from Task 4)
IPC\$	Remote IPC

All shares identified are administrative (hidden) shares created automatically by Windows, denoted by the trailing dollar sign which suppresses their display in standard share browsers. These shares are accessible only to members of the Administrators group by default. The presence of E\$ and F\$ confirms the volumes created during Tasks 2 and 4 are exposed at the share layer. No user-created shares were observed; in a full investigation, any non-administrative shares would be examined closely as they are common mechanisms for data exfiltration or lateral movement, and Get-SmbShareAccess would be used to obtain share ACLs to confirm no permissions have been broadened beyond default.

All Task 6 evidence was collected read-only through standard Windows enumeration cmdlets, in accordance with ACPO Principle 1 (ACPO, 2012) and ISO/IEC 27037:2012 guidance on volatile evidence acquisition (ISO, 2012).

References

1. ACPO (2012) *Good Practice Guide for Digital Evidence*. Association of Chief Police Officers. Available at:
https://www.digital-detective.net/digital-forensics-documents/ACPO_Good_Practice_Guide_for_Digital_Evidence_v6.pdf
2. Brezinski, D. and Killalea, T. (2002) *Guidelines for Evidence Collection and Archiving*. RFC 3227. Internet Engineering Task Force. Available at:
<https://www.rfc-editor.org/rfc/rfc3227>
3. Carrier, B. (2005) *File System Forensic Analysis*. Addison-Wesley Professional.
4. Carvey, H. (2018) *Windows Forensics Cookbook*. Packt Publishing.
5. Casey, E. (2011) *Digital Evidence and Computer Crime: Forensic Science, Computers and the Internet*. 3rd edn. Academic Press.
6. FSR (2023) *Codes of Practice and Conduct for Forensic Science Providers and Practitioners in the Criminal Justice System*. Forensic Science Regulator. Available at:
<https://www.gov.uk/government/publications/forensic-science-providers-codes-of-practice-and-conduct-2021>
7. IEEE (2024) *IEEE Standards for Local and Metropolitan Area Networks: Overview and Architecture*. IEEE.
8. ISO (2012) *ISO/IEC 27037:2012 — Guidelines for Identification, Collection, Acquisition and Preservation of Digital Evidence*. International Organization for Standardization.
9. ISO (2017) *ISO/IEC 17025:2017 — General Requirements for the Competence of Testing and Calibration Laboratories*. International Organization for Standardization.
10. Ligh, M.H. et al. (2014) *The Art of Memory Forensics: Detecting Malware and Threats in Windows, Linux, and Mac Memory*. Wiley.
11. Nelson, B., Phillips, A. and Steuart, C. (2018) *Guide to Computer Forensics and Investigations*. 6th edn. Cengage Learning.
12. Nelson, B. et al. (2008) *Guide to Computer Forensics and Investigations*. 4th edn. Cengage Learning.